



SSL

Índice

Índice	1
Unidad N° 1: Introducción a la Teoría de la Computación	5
Teoría de Autómatas y Lenguajes	5
Historia y concepto de máquinas abstractas y gramáticas formales.	5
Características de las distintas máquinas abstractas.	5
Clasificación de las máquinas abstractas:	5
Jerarquía de máquinas y gramática, y resumen de su vínculo.	6
Utilidad de las máquinas abstractas y sus aplicaciones.	8
Las máquinas abstractas se utilizan para:	8
Compiladores	9
Conceptos de compiladores e intérpretes	9
Estructura y componentes del compilador	9
Contexto de un compilador	11
Tipos de compiladores	12
Identificación y manejo de errores	13
Clases de errores	13
Unidad N° 2: Gramáticas y Lenguajes Formales	14
Introducción a la Lingüística Matemática	14
Símbolos	14
Alfabetos	14
Palabras y lenguajes, y sus operaciones	14
Operaciones con lenguajes	15
Descripción de lenguajes	15
Reglas de reescritura o producciones	16
Derivaciones y reducciones	16
Determinación de lenguajes	16
Gramáticas Formales	16
Definiciones	16
Tipos de gramáticas	17
Jerarquía de Chomsky	17
Lenguajes regulares	18
Gramáticas regulares	18
Definición de gramática regular a partir de AFD	18
Definición de autómatas a partir de gramática regular	18
Conversión de gramática regular por izquierda a regular por derecha	19
Expresiones regulares	19
Método o algoritmo de Thompson:	19
Lenguajes independientes del contexto	21
Gramática limpia	21
Gramática bien formada	21

Análisis Sintáctico	22
Concepto	22
Árboles de derivación	22
Ambigüedad	22
Recursividad	22
Factorización	23
Formas normales de Chomsky y de Greibach; conversiones.	24
FNC	24
FNG	24
Unidad Nº 3: Máquinas Secuenciales y Autómatas Finitos Deterministas	26
Conceptos de máquinas secuenciales y autómatas	26
Máquinas secuenciales de Mealy y Moore	26
Definición de AFD, representación	27
Configuración	27
Extensión al tratamiento de palabras	28
Aceptación de lenguajes	28
Equivalencia de estados	29
Minimización de autómatas	29
Autómatas finitos bidireccionales	30
Aceptación de cadenas	31
Unidad Nº 4: Autómatas Finitos No Deterministas	32
Autómatas finitos no deterministas	32
Definición	32
Transiciones Lambda	32
Equivalencia entre AFD y AFND	33
Teorema 1:	33
Teorema 2:	33
Entre AFND y gramáticas regulares, entre AFND y expresiones regulares	34
Algoritmo de Thompson	35
Unidad Nº 5: Autómatas con Pila	36
Autómatas con Pila	36
Definición	36
Funcionamiento	36
Configuración y movimientos	37
Autómatas con pila deterministas y no deterministas	37
Lenguaje aceptado por el autómata con pila y formas en los que lo hace	37
Analizadores Sintácticos	38
Enfoque descendente y ascendente	38
ANALIZADOR SINTÁCTICO DESCENDENTE (ASD – PDA(LL)):	38
Unidad Nº 6: Autómatas Linealmente Acotados, Máquina de Turing y Complejidad	40
Autómata linealmente acotado y máquina de Turing	40
Definición	40

Funcionamiento	40
Aceptación o reconocimiento de cadenas	40
Configuración o descripción instantánea	41
Si $qn \in A$ y $m = an + 1$, ésta ES una configuración de aceptación.	41
Máquina de Turing	41
$(\Sigma, \Gamma, Q, q_0, A, f, b)$	41
FUNCIONAMIENTO	42
ACEPTACIÓN O RECONOCIMIENTO DE CADENAS	42
CONFIGURACIÓN O DESCRIPCIÓN INSTANTÁNEA	42
Variantes	43
Modularizada	43
Universal	43
Máquina de Turing que puede leer otra MT convenientemente codificada en su cinta y la entrada de ésta, para emular su funcionamiento.	43
Generalizada	43
No determinista	43
Complejidad Computacional	44
Concepto	44
Complejidad de los problemas y de las soluciones	44
Métricas: complejidad estructural, espacial y temporal de la máquina de Turing	44
Unidad Nº 7: Simuladores de Máquinas Abstractas	46
Simulación y otros conceptos relacionados	46
Arquitectura de los simuladores	46
Editor	46
Generador de movimientos	46
Gestor de representación	46
Simulación de Máquinas Abstractas	46
Selección y evaluación de simuladores	48
Requerimientos funcionales de un SMA	48
Requerimientos no funcionales de un SMA	48
Interfaz gráfica (GUI)	49
Unidad Nº 8: Introducción a la semántica de lenguajes	50
Semántica de Lenguajes	50
Concepto	50
Semántica natural	50
Especificación formal	50
Operacional	50
Denotacional y axiomática	51
Gramáticas con atributos	51
Atributos	52
Reglas semánticas	53
Atributos heredados y sintetizados	53
Atributos estáticos y dinámicos	53

Cuadro resumen máquinas abstractas	53
Anexo: Bibliografía y Links	54
Modalidad Académica	54
Bibliografía	54

Unidad N° 1: Introducción a la Teoría de la Computación

Teoría de Autómatas y Lenguajes

Historia y concepto de máquinas abstractas y gramáticas formales.

Una Máquina abstracta, es un dispositivo formal capaz de exhibir conductas que han sido predeterminadas. Se llaman abstractas porque no necesariamente existen como objetos materiales. Se usa para el diseño, implementación y evaluación de algoritmos, análisis de su complejidad, evaluación de los recursos necesarios para determinado cómputo, simulación de su comportamiento sobre nuevos microprocesadores, inclusive aún sin construirlo, y la identificación de eventuales conflictos en procesos concurrentes o paralelos. Estas máquinas también suelen llamarse Máquinas de Estado, ya que son sistemas reactivos que operan en respuestas a los sucesivos estímulos del exterior, lo que lleva a la máquina adoptar un estado y eventualmente enviar una respuesta. Estas máquinas poseen ciertas características comunes:

- Reconocen el paso del tiempo en forma discreta (en intervalos)
- En cada intervalo, se encuentra en un y único estado, siendo el conjunto de sus estados finitos agrupados en un alfabeto de estados
- Recibe el estímulo, lee el símbolo de entrada y el conjunto de todos los símbolos reconocidos por la máquina, forma un alfabeto de entrada.
- Una función de transición define para cada símbolo de entrada y para cada estado posible, el próximo estado a ser adoptado.

Gramática Formal, es un conjunto de reglas para escribir cadenas de caracteres, junto con un símbolo inicial desde el cual se debe comenzar la reescritura, por ello se piensa como una generadora de lenguajes. También, puede ser usada como reconocedor a través de una función que determina si una cadena cualquiera pertenece a un lenguaje o si es gramaticalmente incorrecto.

Las gramáticas describen y generan a un lenguaje formal, por esta razón podemos decir que una máquina abstracta es el equivalente, ya que por definición reconoce y genera lenguajes formales.

Características de las distintas máquinas abstractas.

Clasificación de las máquinas abstractas

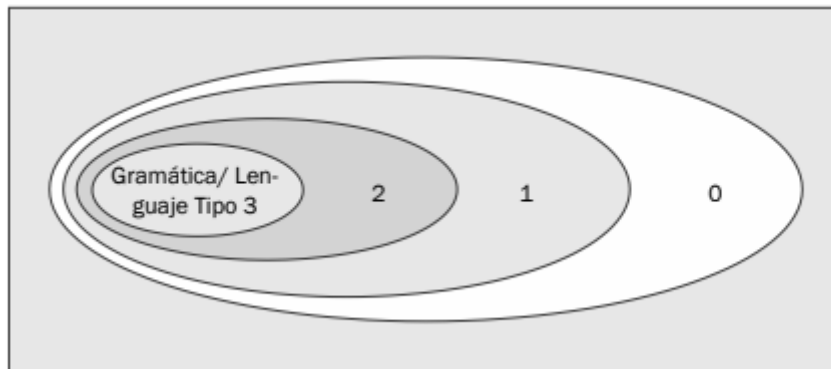
- **Traductoras**: establecen una relación entre las cadenas de entrada y salida.
- **Reconocedoras**: tienen la misión de validar o aceptar ciertas cadenas de entradas, llegando a estados finales definidos como aceptación.
- **Finitos**: comienzan su operación en un estado inicial y completa su ciclo arribando a un estado de aceptación que ha sido aceptado oportunamente identificado.

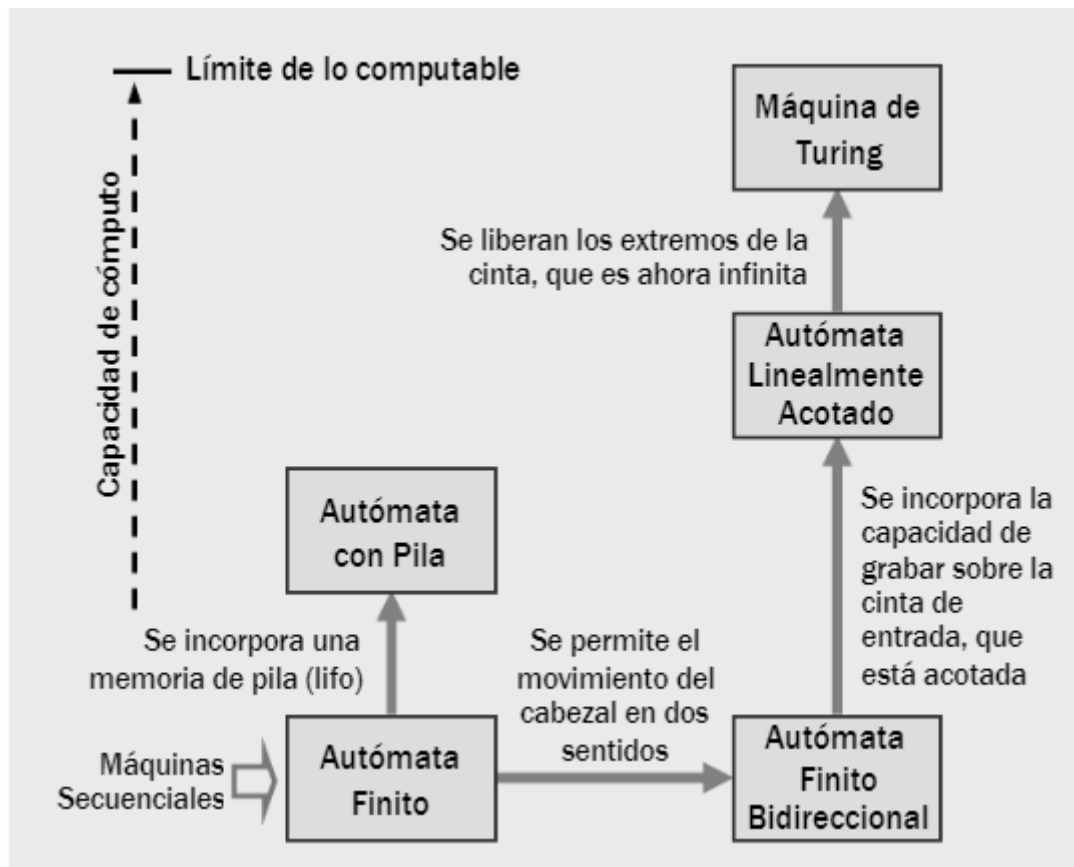
- **Secuenciales:** operan de forma ininterrumpida sin reconocer condiciones específicas para iniciar y terminar su actividad.
- **Determinante:** contemplan un único próximo estado en función de cada estado posible y el símbolo del alfabeto de entrada.
- **No determinante:** pueden tener más de un próximo estado a partir de cada estado y símbolo de entrada, además de poder transitar sin leer el símbolo de entrada.

Jerarquía de máquinas y gramática, y resumen de su vínculo.

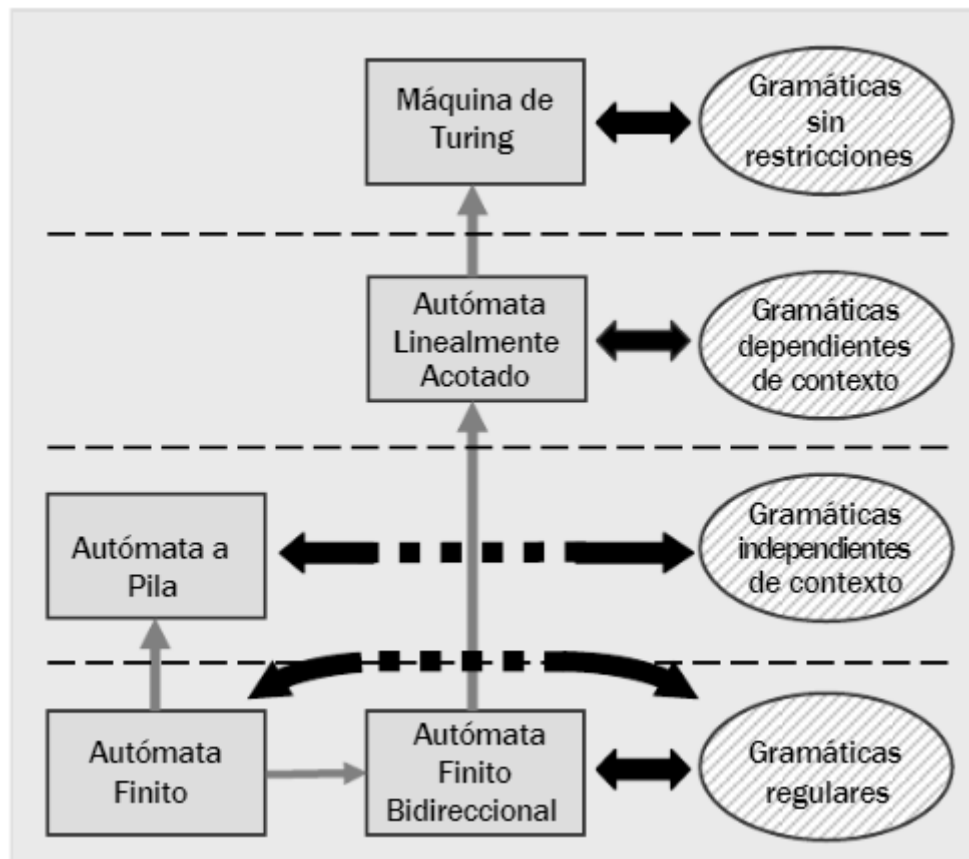
Noam Chomsky, presentó la “Teoría de las gramáticas transformacionales”, donde explica que los lenguajes de computación evolucionaron rápidamente hasta alcanzar un nivel de abstracción capaz de especificar datos, funciones y su control independiente de los computadores a cargo de la ejecución de procesos. Clasifica las gramáticas en 4 tipos:

- Tipo 0: Sin restricciones o de estructuras de fases. Ejemplo las máquinas de Turing.
- Tipo 1: Sensibles al contexto. Ejemplo, Autómata linealmente acotado.
- Tipo 2: Independientes del contexto. Ejemplo, Autómata a pila.
- Tipo 3: Regulares. Ejemplo, Autómata Finito.





Como ya fue anticipado, los autómatas finitos y la máquina de Turing representan los dos extremos de una jerarquía creciente en complejidad y capacidad. Si se comienza por el autómata finito se puede incrementar su capacidad de cómputo incorporándose una memoria LIFO, obteniéndose el autómata con pila. Como alternativa, pueden lograrse soluciones más eficientes permitiendo el movimiento del cabezal de entrada en dos sentidos con el autómata finito bidireccional. Esta máquina, a su vez, mejorará su capacidad de cómputo al poder grabar sobre el medio de entrada, dando lugar al autómata linealmente acotado. Finalmente, esta evolución conduce a la máquina de Turing al liberar los extremos de la cinta y hacerla infinita, todo lo cual se representa en la Figura. Nótese que la máquina de Turing es capaz de resolver todo problema que tenga solución y, por lo tanto, representa el límite natural de lo computable.



En 1958, el propio Chomsky y Miller comprobaron que las gramáticas regulares (Tipo 3) están directamente relacionadas con los autómatas finitos; luego un año después Chomsky estableció la equivalencia entre las gramáticas sin restricciones (Tipo 0) y las máquinas de Turing. Hubo que esperar hasta 1963 para que se demostrara el vínculo entre las gramáticas independientes del contexto (Tipo 2) y los autómatas con pila, por Chomsky y Schutzenberger y, por último, Kuroda, en 1964, descubrió que los lenguajes generados por las gramáticas dependientes del contexto (Tipo 1) son reconocidos por los autómatas linealmente acotados.

Utilidad de las máquinas abstractas y sus aplicaciones.

Las máquinas abstractas se utilizan para:

1. Diseño, implementación y evaluación de algoritmos.
2. Análisis de la complejidad de los algoritmos.
3. Modelar la evaluación de los recursos que necesita un computador.
4. Simulación del comportamiento de nuevos procesadores.
5. Construcción de compiladores.

Compiladores

Conceptos de compiladores e intérpretes

Un **compilador** es una herramienta de desarrollo de software destinada a traducir un programa escrito en lenguaje de alto nivel a otro escrito en lenguaje máquina. Al comienzo eran códigos binarios, representando instrucciones individuales que reconocían el cpu, los cuales fueron reemplazados por los lenguajes de primeras generación, donde el uso era difícil susceptible a errores y mantenimiento costoso. Las abreviaturas mnemónicas, es decir reemplazar el código máquina por nombres simbólicos, se transformaron en el lenguaje ensamblador, que eran de fácil uso y mantenimiento, aunque requería alto conocimiento de las arquitecturas de las máquinas. Por último, el mayor nivel de abstracción, comenzó a ser lenguajes de alto nivel, donde era independiente de la máquina y la programación sufrió una rápida evolución

Intérprete: Se denomina así a una herramienta o módulo que interpreta y ejecuta las sentencias de un programa fuente una tras otra, sin generar un programa objeto.

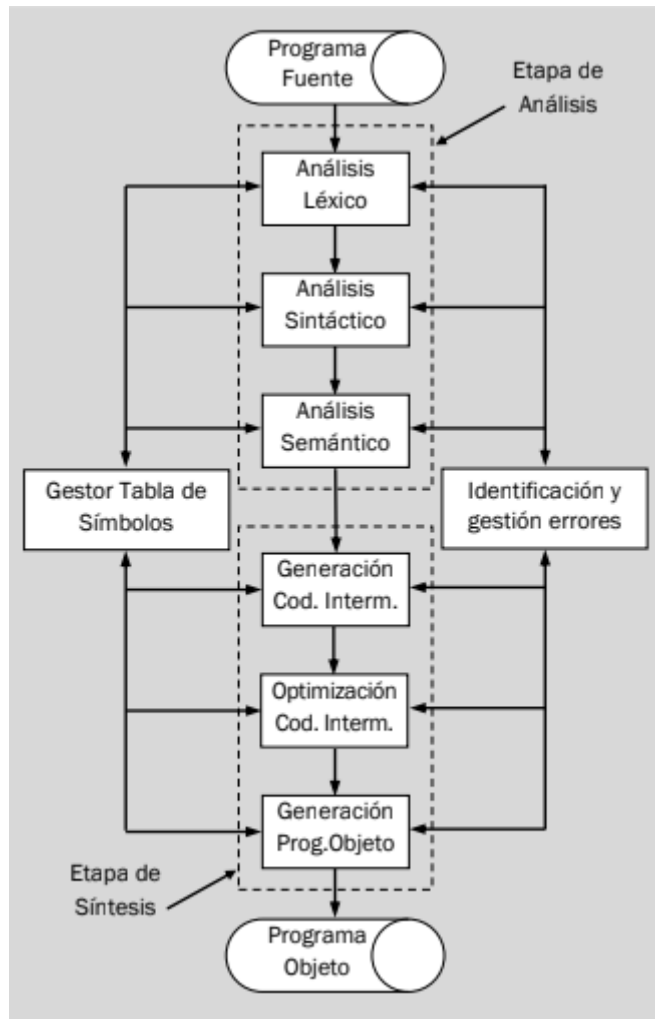
La ventaja de usar intérprete, es leer y ejecutar los programas en forma progresiva sin necesidad de cargar en la memoria el programa fuente completo, lo que lo hace óptimo para cuando se tiene reducida memoria o probar nuevos algoritmos o desarrollar pequeñas aplicaciones.

La ventaja de usar compilador, es que al revisar y tratar el programa fuente en forma íntegra, se logra generar diagnóstico de errores completos que agilizan el desarrollo de programas y evitaban errores de sintaxis o semántica en la ejecución, aunque se necesita más memoria.

Además, los intérpretes se consolidan en las plataformas de desarrollo híbrido, es decir que incluyen etapas de compilación e intérprete, básicamente, por la transportabilidad del software. Y también en las aplicaciones web, dado que en la arquitectura Cliente-Servidor el código intermedio es interpretado directamente en el equipo cliente

Estructura y componentes del compilador

Dentro de la estructura del compilador podemos diferenciar 2 etapas principales, la de análisis y la de síntesis.



Etapas de Análisis: Se divide el programa fuente en sus elementos componentes y se crea una representación intermedia.

1. **Análisis léxico:** Este análisis lo realiza el módulo llamado scanner, leyendo el programa fuente y agrupando léxicos, a través de tokens(cadenas con significado colectivo como variables, operadores aritméticos,etc.), con los lexemas, es decir, caracteres específicos que se ajustan a los patrones léxicos (expresión regular que permite identificar un tipo de token).
2. **Análisis sintáctico:** Existen dos tipos, descendientes y ascendientes, que comprueban que las sentencias sean sintácticamente correctas, y verifican que todas las sentencias han sido generadas por la gramática del lenguaje fuente, sino fue así informa un error de este tipo.
3. **Análisis semántico:** Se revisa el programa fuente para reunir información sobre los tipos de las variables que será usada en la fase posterior, a través de la comprobación estática de tipos flujos de control, unicidad, coherencia en los subprogramas y potenciales errores en tiempo de ejecución. Se busca verificar que lo que está escribiendo tenga sentido a través de métodos.

Etapas de Síntesis: Se reconstruye el programa a partir de esta representación intermedia.

1. **Generación de código intermedio:** se toma como entrada a la representación intermedia y se produce un programa objeto equivalente, que debe ser correcto, eficiente, apropiado al recurso y apto para ejecutarse, a través de la selección del

orden de evaluación de expresiones e instrucciones de la máquina destino, de la asignación de registros, memoria y otros recursos, y la administración de la memoria de para datos y programas.

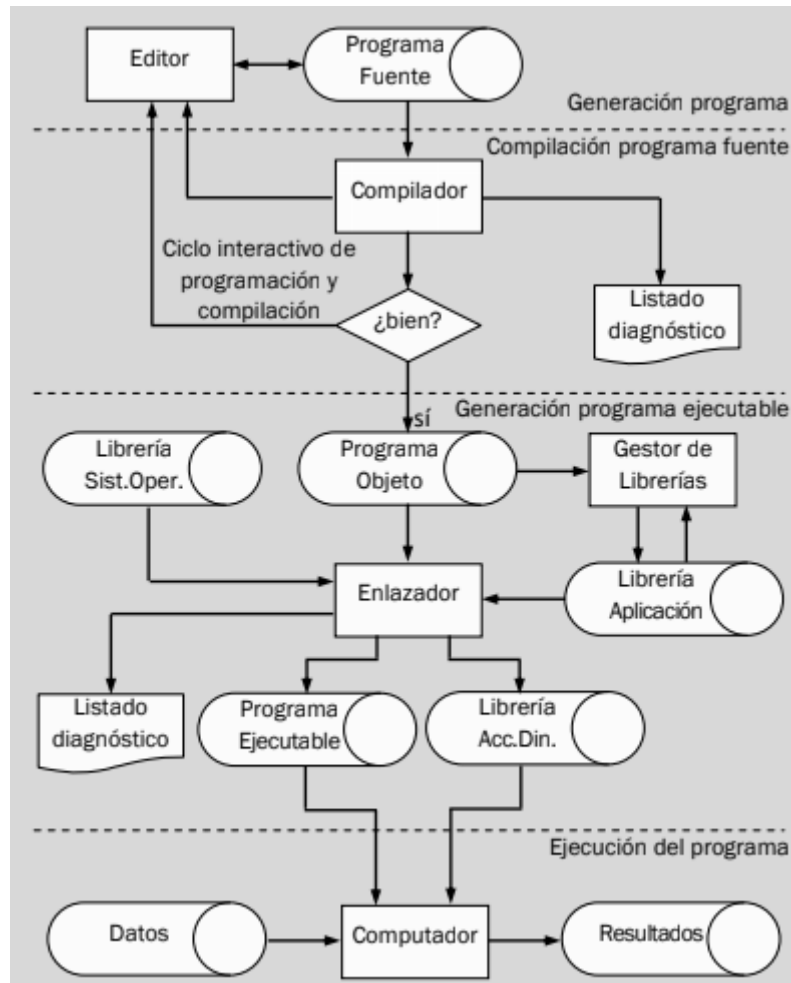
2. Optimización de código intermedio: Se identifican las zonas críticas del programa, es decir, aquellas zonas que por ser ejecutadas reiteradamente afectan significativamente su comportamiento y donde una mejora se verá enormemente potenciada.
3. Generación del programa objeto: Es la fase final del proceso de compilación en la que se toma como entrada la representación intermedia y se produce un programa objeto equivalente.

Gestor de tablas de símbolos: Es un administrador de base de datos que contiene los identificadores del programa fuente y sus atributos como la identidad, tipo, dimensiones, arreglos o direcciones de memoria.

Identificación y gestión de errores: Detecta errores, asociando determinada línea del programa fuente e intenta la recuperación usando un corrector lexicográfico, otro semántico y otro sintáctico.

Contexto de un compilador

Incluye los recursos (módulos de desarrollo) necesarios para hacer posible la generación de programas ejecutables a partir de los correspondientes programas fuentes.



Tipos de compiladores

- **Cruzado:** genera programas fuentes, pudiendo ser ejecutados en distintos computadores
- **Auto Compiladores:** son los que son escritos en el mismo lenguaje que los programas fuentes que admiten, como C
- **Metacompiladores:** son los que admiten programas fuentes en diversos lenguajes, como Meta Assembler
- **Descompilador:** es un módulo que opera en sentido contrario. Por ejemplo los que pasan de máquina a Assembler
- **Optimizador:** tiene como objetivo reducir el tamaño del programa ejecutable, consumo de memoria y aumentar la velocidad de ejecución
- **Gráfico:** son los que admiten programas fuentes representados en forma gráfica, como PLC
- **Intérprete:** genera los programas fuentes en un lenguaje intermedio, como Java
- **IDE (Ambiente integrado de desarrollo):** sistemas interactivos que incorporan al compilador servicios complementarios como un editor, debugging y gestor de librerías.

Identificación y manejo de errores

Verificación: es la comprobación que el producto esté correctamente construido. Puede ser estática (la dinámica, siendo el compilador el encargado de informar clara y oportunamente los errores detectados, donde el IDE y los depuradores ayudan.

Validación: es la comprobación que el producto construido es el correcto

Clases de errores

Estos 3 se detectan con la verificación estática

- **Léxicos:** Símbolos incorrecto en lugares incorrectos
- **Sintácticos:** estructura del programa no respeta las reglas
- **Semánticos:** errores de tipos, argumentos de funciones, etc

Estos 2 se detectan con la verificación dinámica

- Por falla del compilador: excede los límites del compilador
- En tiempo de ejecución: Lógica, periféricos, imprevisiones, etc

Unidad N° 2: Gramáticas y Lenguajes Formales

Introducción a la Lingüística Matemática

Símbolos

Un símbolo es un signo creado convencionalmente, y estos son un concepto abstracto que nos permite hacer referencia a un objeto.

Alfabetos

Se denomina **alfabeto** a cualquier conjunto finito y no vacío de símbolos. En el uso de la materia haremos referencia a ellos utilizando letras griegas en mayúscula $\Sigma, \Gamma, \Lambda, \Omega$

Operaciones entre alfabetos:

- **Igualdad entre alfabetos**, es cuando tienen exactamente los mismos símbolos, son tener en cuenta el orden o repeticiones.
- **Inclusión amplia**, si para todo símbolo perteneciente al primer conjunto, también se encuentran el segundo conjunto.
- **Inclusión estricta**, si es una amplia, pero que exista al menos un símbolo del segundo conjunto que no pertenezca al primero.
- **Cardinal**, es la cantidad de símbolos que tiene ese alfabeto, siendo siempre número entero.
- **Producto cartesiano**
- **Concatenación**, es el conjunto de palabras formadas por la concatenación de símbolos del primer alfabeto con cada uno del segundo. No es generalmente un nuevo alfabeto, sino un conjunto de palabras.
- **Potenciación**, es lo que nos permite obtener un conjunto de palabras de largo n formadas por la misma secuencia de símbolos.
- **Universo de discursos**, es el conjunto de todas las palabras que pueden formarse con sus símbolos, denotado por E^* (Estrella de Kleene del abecedario). También puede ser llamada como clausura, por sus propiedades reflexivas y transitivas.

Palabras y lenguajes, y sus operaciones

Una **palabra** es cualquier secuencia finita de símbolos concatenados, perteneciente a un alfabeto, denotandolas con letras griegas minúsculas.

- **Potenciación de símbolos**: es una notación alterna para escribir una palabra que se forma al concatenar el mismo símbolo
- **Longitud**: es la cantidad de símbolos que conforman una palabra
- **Cadena vacía**: indicada con λ , es aquella palabra que no tiene símbolos, dando de largo 0. Se puede ver que es el equivalente al elemento neutro respecto a la concatenación de palabras.

- **Partes de una palabra:** supongamos que una cadena puede escribirse como concatenación de otras 3. Entonces, tendremos un prefijo, un sufijo y la palabra clave, siendo cualquiera de ellas cadenas vacías. Diremos que un sufijo o prefijo es propio si no es la misma palabra en cuestión o vacía.

El **Lenguaje** es el conjunto de palabras construidas con los símbolos de ese alfabeto, por lo tanto, se le aplica toda la teoría de conjuntos.

$$L_1 \subseteq L_2 \Leftrightarrow \forall \alpha \in \Sigma^*: \alpha \in L_1 \rightarrow \alpha \in L_2$$

“Un lenguaje L_1 definido sobre E es un subconjunto de otro L_2 si toda palabra de L_1 , también es palabra de L_2 ”.

$$L_1 \subset L_2 \Leftrightarrow (\forall \alpha \in \Sigma^*: \alpha \in L_1 \rightarrow \alpha \in L_2) \wedge (\exists \beta \in \Sigma^* / \beta \in L_2 \wedge \beta \notin L_1)$$

“Un lenguaje L_1 definido sobre E es subconjunto de otro L_2 si toda palabra de L_1 es también palabra de L_2 y existe al menos una palabra del segundo que no es elemento del primero”.

$$\begin{aligned} L_1 = L_2 &\Leftrightarrow [\forall \alpha \in \Sigma^*: \alpha \in L_1 \leftrightarrow \alpha \in L_2] \\ &\Leftrightarrow [L_1 \subseteq L_2 \wedge L_2 \subseteq L_1] \end{aligned}$$

“Un lenguaje L_1 definido sobre E es igual a otro L_2 , si tienen exactamente las mismas palabras”

El **conjunto vacío** es un lenguaje llamado lenguaje vacío, tiene cardinalidad 0 y es el único con esta propiedad, independiente del alfabeto sobre el cual está definido.

El **conjunto cuyo único elemento es la cadena vacía** es también un lenguaje único e independiente del alfabeto de definición. Su cardinalidad es igual a 1.

Operaciones con lenguajes

- **Unión:** todas las palabras que pertenecen al menos a uno de los 2 lenguajes.
- **Intersección:** palabras comunes a los 2 lenguajes.
- **Resta:** palabras que pertenecen a uno de los lenguajes pero al otro no.
- **Concatenación:** palabras obtenidas al concatenar cada una de las palabras vacías del primer lenguaje con cada una de las del segundo, en ese orden.
- **Complemento:** todas las palabras del universo del abecedario que no pertenecen al primer lenguaje.

Descripción de lenguajes

- **Extensión:** indicando una por una todas las palabras que son elementos del lenguaje.
- **Comprensión:** explicitando una propiedad de las cadenas sobre algún alfabeto, que solo las palabras del lenguaje verificarán. A su vez, pueden ser coloquialmente,

expresándose en lenguaje natural; conjunto de propiedades, cuando las propiedades se expresan en símbolos; y algebraicamente, cuando la propiedad es expresada usando potenciaciones, uniones, intersecciones, etc.

Reglas de reescritura o producciones

Es un par ordenado de palabras definidas sobre un alfabeto. $\alpha := \beta$ "alfa produce a beta". Una producción constituye un operador unario que puede ser aplicado a una palabra, si y sólo si el lado izquierdo de la producción coincide con una subpalabra de la palabra. El efecto de esto, es obtener una nueva palabra, pero está conformada por beta.

$$\alpha := \beta$$

Derivaciones y reducciones

Se llama derivación directa a la operación que aplica una sola producción a una palabra obteniendo una nueva palabra y se simboliza:

$$\delta \rightarrow \varphi$$

Se llama reducción directa a la operación inversa de una derivación directa.

$$\delta \leftarrow \varphi$$

Determinación de lenguajes

Gramáticas Formales

Definiciones

Gramática formal es una cuadrupla, en la cual sus cuatro componentes representan:

- ΣT Es el alfabeto de Terminales (Símbolos que forman las cadenas del lenguaje).
- ΣNT Es el alfabeto de símbolos No Terminales.
- **S** es un símbolo no terminal distinguido denominado axioma. Siempre es un No Terminal.

- **P** es un conjunto de producciones de la forma “**alfa** := **beta**”, donde ambas palabras están compuestas de símbolos terminales y no terminales pero en alfa al menos debe encontrarse un símbolo no terminal.

$$\Sigma NT \cap \Sigma T = \emptyset$$

Un **lenguaje formal** es un conjunto generado a partir de una gramática formal, de todas las cadenas de símbolos terminales que puedan derivarse desde el axioma **S** usando las reglas de producción **P**.

Una **meta palabra** es una cadena de terminales y no terminales que pueden derivarse desde el axioma de una gramática.

Una **sentencia** es la cadena final de terminales a la que se arriba con la derivación desde el axioma, perteneciente al lenguaje generado por la gramática.

Es muy importante recalcar que dos gramáticas serán **equivalentes** si y sólo si generan exactamente el mismo lenguaje, por lo tanto, el alfabeto de terminales debe ser igual.

Tipos de gramáticas

Jerarquía de Chomsky

Chomsky estableció que todos los lenguajes formales podían clasificarse en 4 tipos diferenciados solo por el formato o “restricciones” de las producciones de las gramáticas que los generan. Estos 4 tipos son los siguientes:

- **Tipo 0 o Estructurado por Frases:** También se les llama lenguajes irrestrictos, ya que son los más generales, estando descritos por reglas de producción menos restrictivas. Las producciones pueden contener cualquier cadena de Terminales o No Terminales en ambos lados, siendo la única condición la existencia de un Terminal a la izquierda. Si en una gramática existe al menos una producción $\Rightarrow$, siendo la cadena del lado izquierdo de mayor longitud que la del lado derecho (regla compresora), entonces la gramática será tipo 0 (salvo la excepción de la regla λ $S \Rightarrow$, que es siempre permitida). Por ejemplo:

$$AZBj \Rightarrow ABj$$

- **Tipo 1 o Dependientes del Contexto:** Permiten el reemplazo contextual de símbolos no terminales. Un símbolo no terminal en el lado izquierdo, sólo puede ser

reemplazado por una cadena de terminales o no terminales en el contexto dado a la izquierda, siendo esta última cadena, al menos de largo unitario. Por lo cuál, la cadena de la izquierda es de largo igual o menor que la de la derecha (regla compresora). Si por el contrario, todas las producciones son no compresoras (salvo la regla lambda) y en ellas siempre un símbolo no terminal del lado izquierdo puede ser reemplazado por una cadena no vacía del lado derecho (dentro o fuera de un contexto), entonces la gramática es dependiente del contexto o tipo 1:

$$ABj := AZBj \text{ o } ABC := AZC$$

- **Tipo 2 o Independientes del Contexto:** se permite reemplazar un símbolo no terminal a la izquierda de la producción, por una cadena de terminales y no terminales en cualquier lugar donde aparezca durante el proceso de derivación, siendo también reglas no compresoras salvo para lambda. Si en una gramática, el lado izquierdo de todas las producciones solo tiene un símbolo no terminal, la gramática es tipo 2 o de contexto libre:

$$A := AZBj \text{ o } B := AZB \text{ o } B := x$$

- **Tipo 3 o Lenguajes Regulares:** Son los lenguajes que tienen producciones más restringidas dentro de la jerarquía de Chomsky, pero de ninguna forma son por ello menos útiles. Finalmente, si todas las producciones de la gramática tienen un no terminal en el lado izquierdo y solo un terminal o un símbolo terminal y un no terminal en el lado derecho, la gramática será regular o de tipo 3:

$$A := 0B \text{ o } B := 1A \text{ o } C := 1$$

Lenguajes regulares

Gramáticas regulares

Definición de gramática regular a partir de AFD

El autómata debe ser un AFD, y la gramática debe ser bien formada y regular por derecha.

Dado $AFD = (\Sigma, Q, q_0, A, f)$

$GR = (\Sigma, Q, S, P) \quad F = P = \{ X \Rightarrow aY \mid f(X, a) = Y \} \cup \{ X \Rightarrow a \mid f(X, a) = Y, Y \in A \}$

Y se agrega $q_0 := \lambda$ si $q_0 \in A$; $X, Y, q_0 \in Q, a \in \Sigma, A \subseteq Q$.

Definición de autómata a partir de gramática regular

Dado: $G = (\Sigma, \Sigma_N, S, P)$

$AF = (\Sigma, \Sigma_N \cup \{A\}, S, \{A\}, f)$

Donde A es un nuevo símbolo que no estaba en Σ_N y $f(X, a) = Y$ si $X \Rightarrow aY$ está en P y $f(X, a) = A$ si $X \Rightarrow a$ está en P, con $X, Y, S \in \Sigma_N, a \in \Sigma$.

La regla de producción $S \Rightarrow \lambda$ incorpora la transición $f(S, \lambda) = A$.

Conversión de gramática regular por izquierda a regular por derecha

Paso 1: Convertir la gramática dada a otra equivalente que no sea recursiva en el axioma. Para ello, se debe incorporar un nuevo símbolo no terminal B y modificar las reglas de producción según se describe:

- Por cada regla $S \Rightarrow \alpha$, se crea una nueva regla $B \Rightarrow \alpha$.
- Cada regla de la forma $A \Rightarrow Sa$ debe ser reemplazada por $A \Rightarrow Ba$ donde $S, A, B \in \Sigma N$, $a \in \Sigma T$ y α es una cadena válida.

Paso 2: Se debe construir un grafo con las siguientes instrucciones:

- Los nodos del grafo se identifican con los símbolos no terminales de la gramática y se incluye un nodo adicional identificado con λ .
- Por cada regla de la forma $A \Rightarrow Ba$ se dibuja un arco desde el nodo A al nodo B que es identificado con la etiqueta a .
- Por cada regla de la forma $A \Rightarrow a$, se incluye un arco desde el nodo A al nodo λ identificado con a .
- Para la regla $S \Rightarrow \lambda$ se incorpora un arco sin etiqueta desde el nodo S al nodo λ .

Paso 3: Se construye un nuevo grafo a partir del grafo anterior según se indica:

- Se intercambian los identificadores de los nodos S y λ
- Se cambia el sentido de todos los arcos

Paso 4: A partir del nuevo grafo, se obtiene la gramática lineal por derecha siguiendo los siguientes pasos:

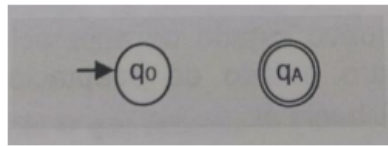
- Los alfabetos de símbolos terminales y no terminales son los mismos de la gramática original lineal por izquierda
- Por cada arco dirigido etiquetado con a desde el nodo B al nodo A , se incorpora a la nueva gramática la regla de producción $B \Rightarrow aA$
- Por cada arco dirigido etiquetado con a desde el nodo B al nodo λ , se incorpora la regla de producción $B \Rightarrow a$
- En caso de un arco sin etiqueta desde el nodo S al nodo λ , se incorpora a la gramática la regla $S \Rightarrow \lambda$

Expresiones regulares

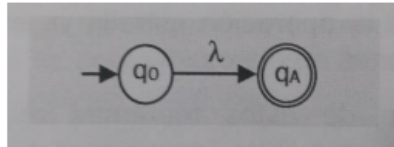
Método o algoritmo de Thompson:

El método que se presenta traduce cada parte de la definición de una expresión regular en un autómata que acepta el mismo lenguaje denotado por ella, de tal forma que el autómata finito quede definido al analizar la expresión parte por parte.

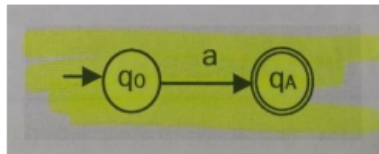
a. Para $\emptyset$:



b. Para λ :

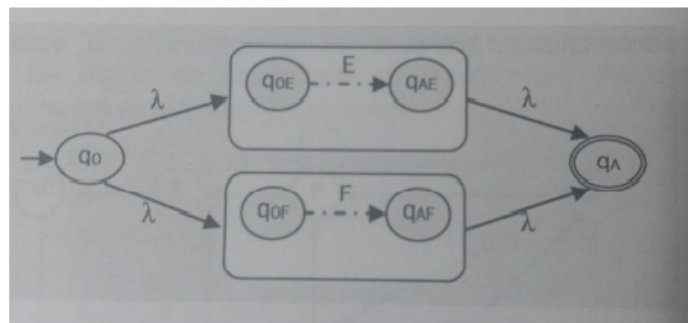


c. Para $a \in \Sigma_E$:

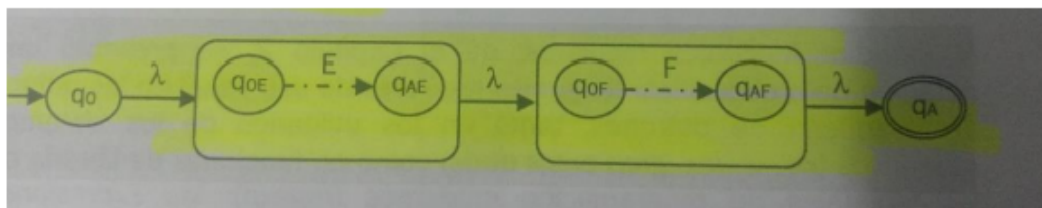


d. El lenguaje unión $L(E)$ y $L(F)$ denotado por la expresión regular $E+F$, será reconocido por el autómata construido de la siguiente forma:

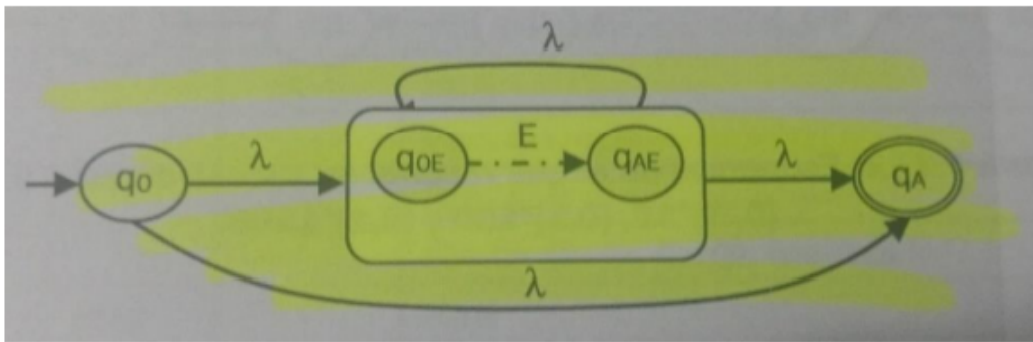
- Quitar la característica de inicial de los estados iniciales de AFe y AFf .
- Crear un nuevo estado inicial q_0 y relacionarlo mediante transiciones λ con antiguos estados iniciales AFe y AFf .
- Quitar característica de aceptación.
- Crear nuevo estado de aceptación q_A y agregar desde cada antiguo estado de aceptación de AFe y AFf una transición λ hacia él.



e. De forma análoga (pero en vez de paralelo, en serie) podemos reconocer $E.F$:



f. Finalmente, para la estrella de Kleene de una expresión regular E^* , que representa al lenguaje $[L(E)]^*$, se construye el siguiente autómata finito.



Lenguajes independientes del contexto

La mayoría de los lenguajes de programación se describen con gramáticas independientes del contexto:

$$S := \lambda \text{ ó } A := \alpha A \in \Sigma N, \alpha \in (\Sigma T \cup \Sigma N)^+$$

Puede verse que el símbolo no terminal A, puede ser reemplazado por la cadena α de terminales y no terminales en cualquier lugar donde aparezca durante el proceso de derivación, sin tener en cuenta el contexto donde se encuentra. Nótese que estas reglas también son compresoras, salvo la regla de lambda que tiene idéntica justificación que en el tipo 1.

Gramática limpia

Para llegar a una gramática limpia se verifica que no haya:

- Reglas innecesarias: es una regla de tipo $A := A$.
- Símbolos inaccesibles: símbolo que no puede ser alcanzado desde el axioma por ninguna derivación válida.
- Símbolos superfluos: símbolo no terminal que no permite generar desde él al menos cadena vacía o de solo símbolos terminales.

Gramática bien formada

Sea la regla $A := \lambda$ no siendo A el axioma, es una regla compresora (no pueden existir reglas compresoras en gramáticas independientes del contexto). Para quitar una regla no generativa:

- Para cada producción $X := \alpha A \beta$ que contenga el no terminal A en el lado derecho, agregar la regla de reescritura $X := \alpha \beta$ que se obtiene de reemplazar A por la cadena vacía.
- Luego eliminar del conjunto de producciones $A := \lambda$, ya que todos los efectos que produciría la misma, han sido incluidos explícitamente como producciones en el paso anterior.

Si tenemos producciones del tipo $A \Rightarrow B$ donde A y B son símbolos no terminales, esta producción, llamada regla de red denominación dice que el no terminal A puede ser reescrito como B en cualquier contexto donde se encuentre. Para eliminar estas reglas:

- Por cada regla $B \Rightarrow \alpha$ existente en la gramática, agregar una regla $A \Rightarrow \alpha$ lo cual hace explícita como producción la posible derivación en dos pasos $A \rightarrow B \rightarrow \alpha$
- Luego puede eliminarse $A \Rightarrow B$ del conjunto de producciones y la gramática obtenida será equivalente a la original.

Análisis Sintáctico

Concepto

$\alpha \in L(G)$?

La manera de responder esta pregunta es encontrando una derivación tal que $S \rightarrow^* \alpha$ que, aplicando una cantidad finita de producciones de la gramática, logre transformar el axioma de la misma en la cadena de terminales bajo análisis.

Entonces podemos decir, que el análisis sintáctico de la cadena α al proceso de búsqueda de esta derivación.

Árboles de derivación

Es una forma pictórica de representar una derivación, iniciándose siempre desde la izquierda partiendo del axioma aplicando al mismo una producción que lo tenga como lado izquierdo. De esta manera, podemos determinar que el árbol tendrá:

- El axioma S de la gramática como raíz.
- Símbolos no terminales de Σ_N como nodos internos.
- Para el nodo interno del no terminal A, si $A \Rightarrow a_1 a_2 \dots a_k$ es la producción usada, se tendrán k nodos hijos etiquetados con los símbolos del lado derecho en el orden en el que aparecen.
- Símbolos terminales de Σ_T como hojas.

Ambigüedad

Se dice que una cadena es ambigua si puede ser generada por derivaciones que admiten distintos árboles de análisis sintáctico.

Recursividad

Una producción de una gramática independiente del contexto se dice que es recursiva si el no terminal de su lado izquierdo se encuentra también del lado derecho $A \Rightarrow \alpha A \beta$

Una gramática que tiene una regla de reescritura recursiva, se dice que tiene recursión en un paso. En el caso en el que un no terminal del lado izquierdo de una producción pueda derivarse en una cadena que lo contenga en varios pasos: $A \rightarrow \delta_1 \rightarrow \delta_2 \rightarrow \dots \rightarrow \alpha A \beta$ se dice que existe en la gramática recursión en más de un paso.

Si en la regla recursiva $A = \alpha A \beta$ la cadena α es vacía, esto es $A = A \beta$, se dice que la regla es recursiva por izquierda y que la gramática tiene recursión izquierda. Si en cambio es β la cadena vacía $A := \alpha A$ se dice que la regla es recursiva por derecha y que la gramática tiene recursión derecha.

Es menester eliminar la recursión izquierda modificando la gramática, pero haciendo que siga generando el mismo lenguaje

Eliminación de la recursión por izquierda en un paso:

Siendo $A := A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$ se puede obtener una gramática equivalente sin recursión izquierda haciendo lo siguiente:

1. Crear un nuevo símbolo no terminal X y agregarlo al alfabeto de símbolos no terminales: $\Sigma N' = \Sigma N \cup \{X\}$.
2. Eliminar todas las producciones en P para el no terminal A .
3. Agregar al conjunto P las producciones donde se reemplazó X .

Eliminación de recursión izquierda en más de un paso:

1. Asignar un orden cualquiera a los símbolos no terminales, digamos $A_1, A_2, \dots, A_k$
2. Para cada $i = 1, 2, \dots, k$ hacer:
 - a. Para cada $j = 1, 2, \dots, k$ hacer: Si $i \neq j$, reemplazar cada $A_i := A_j \delta$ en P (eliminarla y agregar) por $A_i := \gamma_1 \delta \mid \gamma_2 \delta \mid \dots \mid \gamma_m \delta$ donde los γ_m son los lados derechos de todas las producciones de A_j .
 - b. Eliminar recursión izquierda en un paso de A_i , si la hubiere, aplicando el anterior teorema.

Factorización

Factorización por izquierda

Es el caso de dos o más producciones que inicien con una parte común: $A := \alpha \beta$ y $A := \alpha \gamma$

En este caso el proceso de análisis sintáctico no tiene en claro cuál de las dos producciones utilizar durante una derivación, aun sabiendo que tiene que emparejar una parte de la cadena bajo análisis con α . Sin embargo, si se crea un nuevo no terminal X y se reemplazan las anteriores producciones por: $A = \alpha X$ y $X = \beta \mid \gamma$ procedimiento ahora no tiene dudas de lo

que debe utilizar, la primera producción primero, y recién al terminar de emparejar α con la cadena bajo análisis, deberá decidir con cuál de las producciones de X continúa el análisis de la cadena bajo estudio.

FACTORIZACIÓN POR IZQUIERDA

Si en una GIC para un mismo no terminal A , hay producciones que en el lado derecho de la regla de producción poseen los mismos símbolos mas a la izquierda :

$$A := \alpha\beta \mid \alpha\varnothing$$

El proceso de AS no tiene claro cuál camino seguir,



gramática equivalente al reemplazar esas producciones por:

$$A := \alpha X \quad ; \quad X := \beta \mid \varnothing$$

donde X es un nuevo símbolo no terminal,
Proceso intuitivo,
el problema se traslada a X

Formas normales de Chomsky y de Greibach; conversiones.

FNC

Una gramática se dice que está en FNC si y sólo si, todas sus producciones tienen en el lado derecho dos símbolos no terminales, o un solo símbolo terminal o la cadena vacía:

$$A := BC \text{ ó } A := a \text{ ó } S := \lambda$$

Para llevar una gramática a FNC tenemos que proceder con los siguientes pasos:

1. Transformar G en una gramática bien formada, esto es, limpia y sin reglas no generativas ni de red denominación.
2. Para cada símbolo terminal $a \in \Sigma T$, crear un nuevo símbolo no terminal X_a y una nueva producción $X_a := a$
3. Para cada producción de la gramática que contenga en su lado derecho tanto símbolos no terminales como símbolos terminales, reemplazarla por una nueva que tenga en su lugar del terminal a su correspondiente nuevo terminal X_a , es decir $A := \alpha a \beta \rightarrow A := \alpha X_a \beta$. Al terminar con estos pasos se tendrá una gramática equivalente con producciones que solo contienen en su lado derecho un solo símbolo terminal (es decir, que es está en FNC) o una cadena de dos o más símbolos no terminales. Si tiene dos no terminales, entonces ya está en FNC, caso contrario:
4. Para cada producción con más de dos símbolos no terminales en su lado derecho, digamos $A := B\eta$ donde η contiene dos o más no terminales, crear un nuevo símbolo no terminal X y reemplazar la producción por el par $A := BX$ y $X := \eta$.

FNG

Una gramática independiente del contexto está en FNG si y solo si todas sus producciones inician su lado derecho con un símbolo terminal al que le sigue, opcionalmente, una cadena de símbolos no terminales de cualquier largo:

$$A := a\eta \text{ ó } S := \lambda$$

Para llegar a FNG:

1. Transformar G en una gramática bien formada.
2. Quitar la recursividad izquierda de la gramática .
3. Asignar un orden cualquiera a los símbolos no terminales de la gramática .
4. Separar las producciones del conjunto P en tres grupos:
 - a. Grupo 1: todas las producciones que comiencen con un terminal y si existiere en G, la regla lambda $S := \lambda$.
 - b. Grupo 2: producciones $A_i := A_j\alpha$ y con el símbolo A_i anterior a A_j en el ordenamiento dado ($i < j$).
 - c. Grupo 3: producciones $A_i := A_j\alpha$ y con el símbolo A_i posterior a A_j en el ordenamiento dado ($i > j$).
 - i. El caso $i = j$ no puede darse porque se ha eliminado la recursión por izquierda anteriormente..
5. Para cada producción del tercer grupo $A_i := A_j\alpha$ iniciando por aquellas con el subíndice i más pequeño, reemplazarlas por $A_i := \delta_1\alpha|\delta_2\alpha| \dots | \delta_h\alpha$ donde los δ son los lados derechos de todas las producciones de A_j . Al terminar este proceso, todas las producciones pertenecerán al grupo 1 o 2.
6. Repetir el proceso anterior para las producciones del segundo grupo. Al terminar, todas las producciones serán del grupo 1, por lo cual todas iniciarán con un símbolo terminal.
7. Para cada símbolo terminal $a \in \Sigma T$ que esté en el lado derecho de las producciones resultantes, pero no al inicio de las mismas, crear un nuevo símbolo no terminal X_a y una nueva producción $X_a := a$.
8. Para cada producción de la gramática que contenga en su lado derecho, luego del primer símbolo terminal, tanto símbolos no terminales como símbolos terminales, reemplazarla por una nueva que tenga en lugar del terminal no inicial a su correspondiente no terminal X_a .

Unidad N° 3: Máquinas Secuenciales y Autómatas Finitos Deterministas

Conceptos de máquinas secuenciales y autómatas

Autómata finito

En su forma más general, el autómata finito es también traductor, generará una cadena de salida, y al completar la lectura de la cadena de entrada arribará a algún estado final. Es por este motivo que en su operación el autómata finito determina un procedimiento efectivo o algorítmico.

Máquinas secuenciales de Mealy y Moore

Se hace referencia a máquinas secuenciales a las máquinas abstractas más simples. Las principales son la máquina de Mealy y la máquina de Moore.

Máquina de Mealy

Tiene 5 componentes y es definida así: $ME = (\Sigma E, \Sigma S, Q, f, g)$ donde:

- ΣE alfabeto de símbolos de entradas.
- ΣS alfabeto de símbolos de salida.
- Q Conjunto finito y no vacío de estados posibles.
- f Función de transición: $f : Q \times \Sigma E \rightarrow Q$.
- g Función de salida $g : Q \times \Sigma E \rightarrow \Sigma S$

La función de transición f tiene la finalidad de definir el próximo estado que adoptará la máquina a partir de su estado actual y cada uno de los posibles símbolos de entrada. De igual forma, la función de salida g define la salida de la máquina a partir de los mismos argumentos. La máquina de Mealy es traductora, lo que significa que establece una relación entre una cadena de entrada y la cadena de salida.

Máquina de Moore

Se diferencia de la máquina de Mealy únicamente en su función de salida, ya que ésta sólo depende del estado actual y no de la entrada en ese instante, es decir:

$$MO = (\Sigma E, \Sigma S, Q, f, g)$$

donde la función de transición f no cambia y en g hay una relación directa entre el estado en cada intervalo de tiempo y el símbolo de salida: $f : Q \times \Sigma E \rightarrow Q$ $g : Q \rightarrow \Sigma S$

Debe recordarse que la máquina de Moore incorpora un retardo entre la entrada y la salida, si en un instante de tiempo t el autómata se encuentra en un estado $qt \in Q$ es:

$$st = g(qt)$$

Y como este último estado fue a su vez alcanzado en una transición anterior:

$$q_t = f(q_{t-1}, e_{t-1})$$

Se puede apreciar la relación directa entre la salida actual y la anterior, que es:

$$s_t = g(f(q_{t-1}, e_{t-1}))$$

Puede demostrarse que para toda máquina de Moore hay una máquina de Mealy capaz de tener el mismo desempeño y recíprocamente. Lo primero es obvio, ya que solo basta plantear una máquina de Mealy que en cada estado prevea la misma salida para todos los símbolos de entrada. Lo opuesto, obtener una máquina de Moore a partir de una de Mealy, no es tan obvio y requiere mayor esfuerzo.

Definición de AFD, representación

Si a la máquina de Mealy, definida previamente como una quintupla, se le incorpora un estado inicial y un conjunto de estados de aceptación, estaremos en presencia de un AFD. El AFD comienza a operar a partir de un estado inicial y tiene conducta traductora, ya que, transforma las cadenas de entrada en cadenas de salida, y completa su operación al terminar de leer su entrada, arribando a un estado que podrá o no ser de aceptación. Se define como:

$$\text{AFDT} = (\Sigma E, \Sigma S, Q, q_0, A, f, g)$$

ΣE : Alfabeto de símbolos de entrada

ΣS : Alfabeto de símbolos de salida

Q : Conjunto finito y no vacío, de estados posibles

q_0 : Estado inicial de operación, $q_0 \in Q$

A : Conjunto de estados de aceptación, $A \subseteq Q$

f : Función de transición, $f: Q \times \Sigma E \rightarrow Q$

g : Función de salida, $g: Q \times \Sigma E \rightarrow \Sigma S$

Si el AFDt se limita a reconocer o validar cadenas, resultan innecesarios su alfabeto de salida y su correspondiente función de salida, por lo que ambas pueden ser eliminadas, dando lugar al Autómata Finito Reconocedor (AFDR), que queda definido como una quintupla:

$$\text{AFDR} = (\Sigma E, Q, q_0, A, f)$$

Configuración

Se define como configuración o descripción instantánea K_t de un autómata finito en un intervalo de tiempo t al par ordenado:

$$K_t = (q, \beta) ; \text{ con } q \in Q, \beta \in \Sigma E^*$$

Donde q representa el estado actual en el que se encuentra y β el sufijo o subcadena de entrada que está pendiente de ser leída. A partir de esta definición, y ante una cadena de entrada α a ser procesada, se puede reconocer la configuración inicial como:

$$K_0 = (q_0, \alpha)$$

De igual manera se define la configuración de aceptación como:

$$K_n = (q_n, \lambda) ; \text{ donde } q_n \in A, n=|\alpha|$$

El tránsito de una configuración a otra es denominado movimiento, es decir que si existe la transición $q=f(p,a)$, el movimiento del estado p al q leyendo el símbolo de entrada a , puede representarse:

$$(p, a\beta) \vdash (q, \beta)$$

El movimiento desde la configuración inicial a la final es:

$$(q_0, \alpha) \vdash^* (q_n, \lambda)$$

$\vdash^*$ equivale a una cantidad finita de movimientos, que es igual al largo de α , de la cadena de entrada. Se reitera que se utiliza λ para indicar que en la cinta de entrada no ha quedado nada que leer.

Extensión al tratamiento de palabras

La función de transición f define el comportamiento de un AFD de cada estado y ante la entrada de cada posible símbolo quedando expresado este comportamiento como la transición a otro estado. Esta función puede ser extendida para describir lo que ocurre a partir de cada estado si en lugar de recibir símbolos aislados se recibe una secuencia concatenada de los símbolos de entrada. La noción de función de transición extendida a $f_e : Q \times E^* \rightarrow Q$, para operar sobre cadenas, puede inducir a partir de un razonamiento muy simple, considerando que la cadena $\alpha=abcde$ puede ser expresada como $\alpha=a\beta$, donde $\beta=bcde$:

$$f_e(q, \alpha) = f_e(q, a\beta) = f_e(f(q, a), \beta)$$

Se obtiene un proceso recursivo que finaliza al completarse la lectura de α , situación en donde ocurre que $f_e(q, \lambda)=q$. Así, se tiene que:

$$f_e(q, \alpha) = f(f(f(f(f(q, a), b), c), d), e)$$

Aceptación de lenguajes

Si en el movimiento desde la configuración inicial hasta la final, representado por $(q_0, \alpha) \vdash^* (q_n, \lambda)$, se verifica que $q_n \in A$, se dice que la cadena α ha sido reconocida o aceptada. El AFD reconocedor llega a la conclusión de aceptar o rechazar la cadena de entrada luego

de efectuar un número finito de transiciones o movimientos a partir de su configuración inicial. Es decir que la aceptación de una cadena por parte de un AFD implica dos condiciones simultáneas: i) la completa lectura de la cadena de entrada α y ii) el arribo a un estado de aceptación. Por el contrario, si $q_n \notin A$ la cadena α habrá sido rechazada.

A partir de lo expuesto, se dice que un lenguaje L es aceptado por un autómata finito M si todas las palabras que lo componen conducen el autómata a estados de aceptación. En símbolos:

$$L(M) = \{\alpha / \alpha \in \Sigma^* \text{ y } f(q_0, \alpha) \in A\}$$

Autómatas conexos:

Si todos los estados de un autómata son accesibles desde su estado inicial se dice que el mismo es conexo:

$$\text{Un AFD es conexo} \leftrightarrow \forall p_i \in Q: p_i A q_0$$

Equivalencia de estados

Dos estados de un AFD se dice que son equivalentes, lo que es representado por pEq , si desde cualquiera de ellos se aceptan y rechazan exactamente las mismas cadenas de símbolos de entrada.

En símbolos:

$$pEq \leftrightarrow [\forall \alpha \in \Sigma^*: f(p, \alpha) \in A \leftrightarrow f(q, \alpha) \in A]$$

Se reconoce entonces que entre dos elementos p y q del conjunto de estados posibles Q , se ha establecido una relación de equivalencia E , que satisface las propiedades reflexiva, simétrica y transitiva en Q .

Equivalencia entre autómatas:

Dos AFD denominados A_1 y A_2 , son equivalentes (A_1EA_2), si ambos aceptan las mismas palabras o sentencias. Dos AFD, son equivalentes si y sólo si aceptan el mismo lenguaje.

$$A_1EA_2 \leftrightarrow L(A_1)=L(A_2)$$

Minimización de autómatas

Autómata mínimo

Autómata que cumple correctamente su función con la menor cantidad posible de estados. Para minimizar un autómata se recurre al concepto de conjunto cociente visto con anterioridad, es decir, a la partición del conjunto de estados Q producida por la relación de equivalencia E , que tiene por elementos a las clases de equivalencia, donde se agrupan todos los estados equivalentes o indistinguibles entre sí. El autómata mínimo equivalente a uno dado tendrá como estados a cada una de las partes o clases de estados equivalentes del autómata original (es decir, su conjunto de estados será Q/E), aquella clase que contenga el estado inicial original será su estado inicial, las que contengan algún estado de

aceptación del autómata original serán sus estados de aceptación; su función de transición se construirá en base a la del autómata original.

Autómatas finitos bidireccionales

Es un AFD con la capacidad de decidir el sentido del movimiento del cabezal en cada intervalo de tiempo. En la definición de autómata se debe incorporar una función de movimiento. Se incorporan además, dos símbolos especiales que identifican el inicio y el final del intervalo de cinta de entrada a ser analizado.

Tiene las siguientes características

:

- Cada símbolo de entrada puede ser leído varias veces.
- No hay ninguna condición que identifique el fin de la lectura de la cadena.
- No es posible anticipar la cantidad de intervalos de tiempo requeridos para evaluar la cadena.
- El concepto de cadena que resta de ser leída desaparece.

Una consecuencia puede ser que ocurra que ciertas cadenas de entrada lleven al AFBD a un ciclo cerrado que no conduzca a nada, es decir, un ciclo infinito.

Se definen entonces:

$$AFBD = (\Sigma E, \Gamma, Q, q_0, A, f)$$

Donde:

- Γ Alfabeto de cinta $\Gamma = \Sigma E \cup \{\vdash, \dashv\}$
- f Función de transición $f: Q \times \Gamma \rightarrow Q \times \{I, N, D\}$

Sobre la definición de AFDB nótese que:

1. El alfabeto de cinta Γ se forma con el alfabeto de entrada y dos símbolos auxiliares destinados a marcar el inicio y fin de la cinta de entrada, normalmente denominados BOT (Beginning of tape $\vdash$) y EOT (End of tape $\dashv$) estos símbolos nunca son de entrada.
2. El AFDB tiene normalmente un único estado de aceptación ya que al poder ser releída la cadena siempre podrá ser definido con uno solo de estos estados.
3. La función de transición es definida usando el alfabeto ampliado Γ ya que el autómata debe reconocer todos los símbolos que pueda contener la cinta de entrada y tomar decisiones a partir de ellos
4. La función de transición f reúne dos funciones, la de transición de estado a estado $t: Q \times \Gamma \rightarrow Q$ y la del sentido del próximo movimiento del cabezal de entrada $m: Q \times \Gamma \rightarrow \{I, N, D\}$
5. El alfabeto de movimientos del cabezal contiene tres símbolos, izquierda (I), neutro (N) y derecha (D).

Aceptación de cadenas

Una cadena es aceptada por el AFDB cuando éste ha alcanzado un estado de aceptación y la cadena ha sido leída por lo menos una vez. Esto último significa que la cantidad de intervalos de tiempo demandados por la aceptación será igual o mayor al largo de la cadena en cinta.

Configuración

Para la completa definición de la condición en la que se encuentra un AFDB en un instante dado se requieren 3 componentes:

1. El estado actual.
2. El contenido de la cinta de entrada.
3. La posición del cabezal.

Se adopta además la convención de que el símbolo de inicio de cinta (BOT) está en la posición 0 y el contador se incrementa hacia la derecha.

La configuración kt de un AFDB que opera sobre cierta cadena de entrada α que en el instante de tiempo t está en el estado q y con el cabezal en la posición k es:

$$Kt = (q, \vdash \alpha \dashv, k)$$

Entonces, la configuración inicial será: $K0 = (k0, \vdash \alpha \dashv, 0)$ y la configuración final será: $KA = (qA, \vdash \alpha \dashv, n)$ donde: $qA \in A$ y $n = |\alpha| + 1$.

Lenguaje reconocido por el AFDB

El lenguaje L que es aceptado por un autómata finito bidireccional M puede definirse en símbolos como:

$$L(M) = \{\alpha/\alpha \in \Sigma E * \text{ y } q0\alpha \vdash * \alpha qA, qA \in A\}$$

Unidad N° 4: Autómatas Finitos No Deterministas

Autómatas finitos no deterministas

Definición

Los AFND quedan definidos como una quintupla:

$$\text{AFND} = (\Sigma E, Q, q_0, A, f)$$

Y su característica distintiva está en la definición de las transiciones, que aquí no es más una función de estado a estado, sino una relación de transición, tomando la forma:

$$f: Q \times \Sigma E \rightarrow P(Q)$$

Donde $P(Q)$ es el conjunto de todos los subconjuntos que se pueden formar con los elementos del conjunto Q (conjunto potencia de Q).

Esta indeterminación, no significa incertidumbre o aleatoriedad en el desempeño final del autómata. Más bien significa incertidumbre en cuanto al esfuerzo requerido al evaluar una cierta cadena. Esto es así porque la operación del autómata implica una búsqueda exhaustiva en un árbol de posibilidades que será más frondoso según sea mayor la cantidad de próximos estados posibles en cada transición, llamada **factor de ramificación** FR, donde $FR = |f(q, e)|$ donde $q \in Q$, $e \in \Sigma E$. El tiempo para resolver este tipo de problemas es exponencial.

La flexibilidad que ofrece el no determinismo es de gran ayuda en el diseño de autómatas destinados a reconocer lenguajes complejos. Se obtienen diseños más simples a costa de algoritmos menos eficientes.

Transiciones Lambda

La definición de los AFND puede ampliarse para incluir transiciones de un estado a otro que no dependan de ninguna entrada, las que son denominadas transiciones λ .

Para contemplarlas se incorpora una columna adicional a la relación f a fin de considerar los pares (q, λ) y se asume además que cada estado tiene su propia transición λ que es cíclica

sobre sí mismo. Luego la expresión de f debe ser reescrita de manera de adicionar λ a los símbolos del alfabeto de entrada:

$$f: Q \times (\Sigma E \cup \lambda) \rightarrow P(Q)$$

A todos los pares de estados que están vinculados por una transición λ es conveniente reunirlos en un conjunto T denominado relación de transición λ . Por lo ya anticipado, en esta relación, se incluyen además las transiciones de los estados con ellos mismos, reconociéndose el carácter reflexivo de T :

$$T = \{(p, q) / q \in f(p, \lambda)\} \cup \{(q, q) / q \in Q\}$$

Luego se incorpora al conjunto T nuevos pares ordenados, que se originan en la aplicación de la propiedad transitiva a los pares anteriores:

$$T^* = T \cup \{(p, q) / pTr \wedge rTq\}$$

Equivalencia entre AFD y AFND

Teorema 1:

Todo AFND- λ puede ser redefinido como un AFD equivalente y para ellos se deben eliminar sus transiciones λ . Se procede a redefinir el conjunto de estados de aceptación A' y la función de transición a f' , mediante los siguientes pasos:

1. Para todo par de estados distintos relacionados por transiciones λ (pT^*q , $p \neq q$) y para cada transición $r = f(q, e)$ donde $e \neq \lambda$, es decir, $r = f(f(p, \lambda^*), e)$ se define una nueva transición $r = f(p, e)$. Se incorporan las nuevas transiciones a f .
2. Para todo par de estados distintos relacionados por transiciones λ (qT^*r , $q \neq r$) y para cada $p = f(p, e)$ donde $e \neq \lambda$, es decir $r = f(f(p, e), \lambda^*)$, se define la nueva transición $r = f(p, e)$. Se incorporan las nuevas transiciones en f .
3. En caso de existir una transición λ desde el estado inicial a un estado de aceptación (q_0T^*r , $r \in A$) hacer $q_0 \in A$.
4. Eliminar todas las transiciones λ , lo que implica eliminar la columna λ de la tabla de f .

Teorema 2:

Por cada AFND siempre existe un AFD que acepta el mismo lenguaje y estos dos autómatas se dice que son equivalentes.

1. Reconocer el estado inicial $C_0 = \{q_0\}$.
2. Definir los nuevos estados $C_i = f'(C_k, e)$ para todo $e \in \Sigma E$ y $k < i$.
3. Para todos los casos en que $C_i = \{\dots, q_n, \dots\}$ y $q_n \in A$ se reconoce que $C_i \in A'$.

Isomorfismos

Entre AFND y gramáticas regulares, entre AFND y expresiones regulares

AFD $\rightarrow$ Gramática Regular:

Para cada $AFD = (\Sigma E, Q, q_0, A, f)$ se puede construir una gramática regular $G = (\Sigma T, \Sigma N, S, P)$ tal que $L(AFD) = L(G)$, con el siguiente procedimiento:

1. $\Sigma T = \Sigma$ símbolos terminales de G son de entrada del AF .
2. $\Sigma N = Q$ símbolos no terminales de G son estados de AF .
3. $S = q_0$ el axioma de G es el estado inicial del AF .
4. P se arma desde la función de transición f del AF :
 - a. Si $f(X, a) = Y$ entonces agregar la producción $X: = aY$.
 - b. Si $f(X, a) = Y \in A$ entonces agregar la producción $X: = a$.
 - c. Si $f(S, \lambda) = Y \in A$ entonces agregar la producción $S: = \lambda$.

Gramática Regular $\rightarrow$ AFD:

Para cada $G = (\Sigma T, \Sigma N, S, P)$ se puede construir un autómata finito no determinista $AFND = (\Sigma E, Q, q_0, A, f)$ tal que $L(AFND - \lambda) = L(G)$, con el siguiente procedimiento:

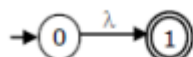
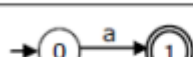
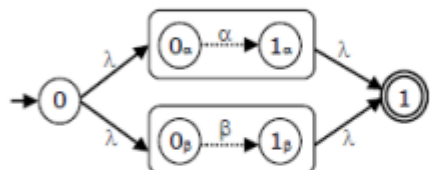
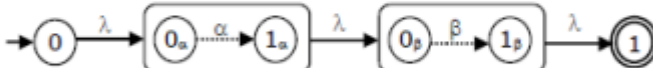
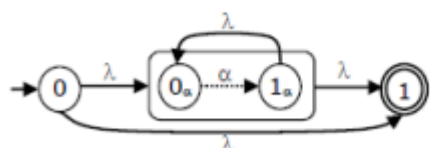
1. $\Sigma = \Sigma T$ símbolos de entrada del AF son terminales de G .
2. $Q = \Sigma N \cup \{F\}$ estados del AF son símbolos no terminales de G más un nuevo estado F que será de aceptación.
3. $q_0 = S$ el estado inicial del AF es el axioma de G .
4. $A = \{F\}$ el conjunto de estados de aceptación tiene solo F .
5. f se arma desde el conjunto de producciones de P :
 - a. Si $X: = aY$ está en P entonces agregar $f(X, a) = Y$.
 - b. Si $X: = a$ está en P entonces agregar $f(X, a) = F$.
 - c. Si $S: = \lambda$ está en P entonces agregar $f(S, \lambda) = F$.

ISOMORFISMO ENTRE EXPRESIONES REGULARES Y AFND- λ :

ER $\rightarrow$ AFND - λ : Para cada expresión regular se puede construir un autómata finito no determinista $AFND - \lambda = (\Sigma E, Q, q_0, A, f)$ tal que $L(AFND - \lambda) = L(ER)$, utilizando el procedimiento denominado Algoritmo de Thompson.

Algoritmo de Thompson

El método que se presenta traduce cada parte de la definición de una expresión regular en un autómata que acepta el mismo lenguaje denotado por ella, de tal forma que el autómata finito quede definido al analizar la expresión parte por parte.

Expresión Regular	Lenguaje Denotado	Autómata finito que reconoce el lenguaje denotado
$\emptyset$	$\emptyset$	
λ	$\{\lambda\}$	
a	{a}	
$\alpha + \beta$	$L(\alpha) \cup L(\beta)$	
$\alpha \cdot \beta$	$L(\alpha) \cdot L(\beta)$	
α^*	$[L(\alpha)]^*$	
(α)	L(α)	

Unidad N° 5: Autómatas con Pila

Autómatas con Pila

Definición

El Autómata con Pila (AP), en inglés Push Down Automata, es básicamente un autómata finito al que se le ha incorporado una memoria de pila. La memoria de pila o memoria LIFO

(Last In First Out) incrementa la capacidad de resolver problemas del autómata finito convencional, al incorporar la posibilidad de memorizar total o parcialmente la cadena leída y cualquier otra marca que ayude al procesamiento de la misma.

El AP queda definido como la séptupla:

$$AP = (\Sigma E, \Gamma, Q, q_0, \#, A, f)$$

Donde:

Γ : es el alfabeto de símbolos de pila

$\#$: Símbolo de referencia de pila, $\# \in \Gamma, \# \notin \Sigma E$

En el caso de un AP no determinista (APND):

$$f: Q \times (\Sigma E \cup \{\lambda\}) \times \Gamma \rightarrow P(Q \times \Gamma^*)$$

Para AP deterministas se presentan dos posibles funciones de transición:

$$f: Q \times \Sigma E \times \Gamma \rightarrow Q \times \Gamma^*$$

$$f: Q \times (\Sigma E \cup \{\lambda\}) \times \Gamma \rightarrow Q \times \Gamma^*$$

Funcionamiento

Iniciando el funcionamiento en el estado inicial q_0 , y con la pila vacía marcada por $\#$, el autómata:

1. Lee un símbolo de entrada $lec = eleido$ o no lee $lec = \lambda$
2. Saca un símbolo de pila $w = sacar()$.
3. Transita a uno o más estados y almacena cadenas en la pila $\{(q_1, \beta_1), (q_2, \beta_2), \dots, (q_m, \beta_m)\} = f(q_{actual}, lec, w)$.
4. Repite los pasos 1-3 en forma no determinística hasta terminar de leer la cadena de entrada.
5. Decide si acepta o rechaza la cadena leída.

Aquí el funcionamiento **es no determinista** de varias formas:

- a) El AP puede leer un símbolo de entrada o no leer, por lo que puede realizar transiciones espontáneas (λ).
- b) El rango de la función de transición es $P(Q \times \Gamma^*)$, por lo que puede transitar a varios estados y por cada estado
- c) Podría realizar sobre la pila una de varias acciones:

1. Desapilar: extrae un símbolo w y no almacena nada.

$$f(p, a, w) = \{(q, \lambda)\}$$

2. Apilar: extrae un símbolo w y lo devuelve con algo más $w\beta$.

$$f(p, a, w) = \{(q, w\beta)\} \text{ con } \beta \in \Gamma^*$$

3. Cambiar: extrae un símbolo w y almacena algo más β .

$$f(p, a, w) = \{(q, \beta)\} \text{ con } \beta \in \Gamma^*$$

Configuración y movimientos

$$kt = (q, \beta, \delta)$$

Donde q representa el estado en el que se encuentra, β la subcadena de entrada pendiente de ser leída y δ el contenido de la pila.

Configuración inicial: $k0 = (q0, \alpha, \#)$

Configuración final: $kn = (q, \lambda, \delta)$

El tránsito de una configuración a otra es también aquí denominado movimiento:

$$(p, a\beta, \delta b) \vdash (q, \beta, \delta bb)$$

El movimiento de configuración inicial a la final es:

$$(q0, \alpha, \#) \vdash * (q, \lambda, \delta)$$

Autómatas con pila deterministas y no deterministas

Una vez que el AP ha completado la lectura de una sentencia, debe verificarse que la pila se encuentra vacía y la única forma es leerla y comprobar que el símbolo leído es precisamente la marca de fondo de pila ($\#$). Por esta razón, cuando los autómatas han completado la lectura de la cadena de entrada, realizan un movimiento adicional que está destinado a verificar la condición de la pila, y en caso de estar vacía, transitar al estado de aceptación. Aquí la cadena de entrada ya ha sido leída y esta lectura adicional de la pila se representa como una transición λ .

Es importante advertir que, pese a la transición λ , esto por sí solo no define al AP como no determinista. En efecto, si se completó la lectura de la sentencia de entrada con el autómata en un cierto estado q , el AP será determinista siempre que:

$$|f(q, \lambda, \#)| = 1 \text{ y } f(q, a, \#) = \emptyset \text{ para todo símbolo } a \in \Sigma E$$

Y será considerado no determinista cuando:

$$f(q, \lambda, \#) \neq \emptyset \text{ y } f(q, a, \#) \neq \emptyset \text{ para algún símbolo } a \in \Sigma E$$

Lenguaje aceptado por el autómata con pila y formas en los que lo hace

1) *Aceptación por vaciado de pila:*

$$L = \{\alpha (q0, \alpha, \#) \vdash * (q, \lambda, \#) \text{ con } q0, q \in Q, \alpha \in \Sigma E^*, \# \in \Gamma\}$$

2) *Aceptación por estado final:*

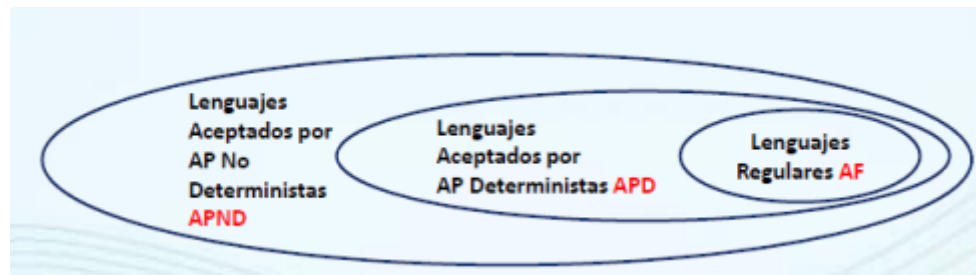
$$L = \{\alpha (q0, \alpha, \#) \vdash * (q, \lambda, \delta) \text{ con } q0 \in Q, q \in A, \alpha \in \Sigma E^*, \delta \in \Gamma^+, \# \in \Gamma\}$$

3) *Aceptación por vaciado de pila y estado final:*

$$L = \{\alpha (q0, \alpha, \#) \vdash * (q, \lambda, \#) \text{ con } q0, q \in Q, q \in A, \alpha \in \Sigma E^*, \# \in \Gamma\}$$

No equivalencia entre los APD y APND

A diferencia de lo que ocurre con los autómatas finitos, el no determinismo otorga a los APND mayor poder de cómputo respecto de los APD. Esto quiere decir que un APND puede reconocer lenguajes que un APD no puede.



Autómatas con pila asociados a una gramática

En el caso de las gramáticas independientes del contexto, estos autómatas llevan el nombre de analizadores sintácticos. La denominación de “compiler-compiler” proviene del hecho que los generadores de analizadores tienen por finalidad la construcción de analizadores sintácticos, los que a su vez forman parte de la etapa de análisis de los compiladores. Esta etapa incluye las fases de análisis léxico, destinado a identificar los componentes del lenguaje, análisis sintáctico, que verifica que la estructura corresponda a las reglas de reescritura de la gramática, y finalmente la fase de análisis semántico que se ocupa del sentido y contenido lógico de las sentencias.

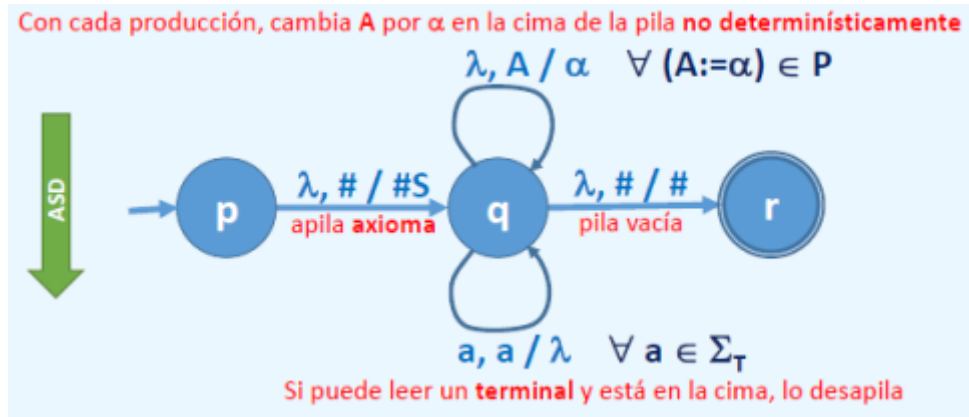
Analizadores Sintácticos

Enfoque descendente y ascendente

ANALIZADOR SINTÁCTICO DESCENDENTE (ASD – PDA(LL)):

A partir del axioma S identifica las producciones de la gramática que sucesivamente deben aplicarse para construir una derivación por la izquierda de la cadena mientras se consume la misma. Genera el árbol desde la raíz hacia las hojas.

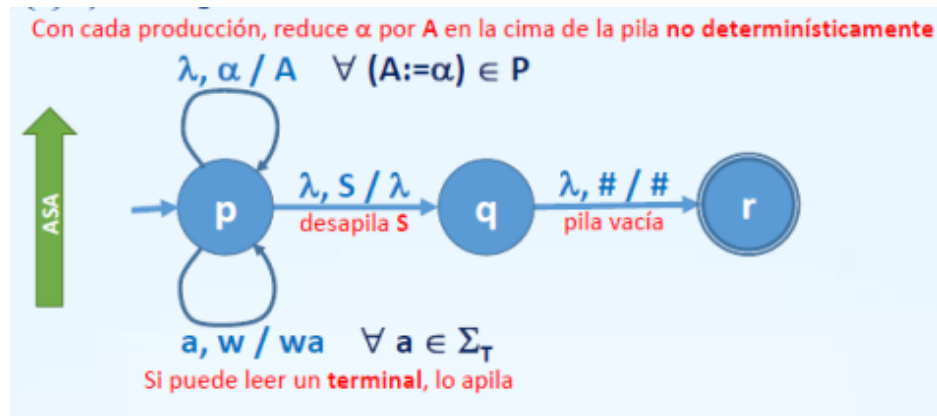
Dada una gramática $GIC = (\Sigma T, \Sigma N, S, P)$ se debe construir un autómata con pila $AP = (\Sigma T, \Sigma T \cup \Sigma N \cup \{\#\}, \{p, q, r\}, p, \#, \{r\}, f)$ de la siguiente forma:



ANALIZADOR SINTÁCTICO ASCENDENTE (ASA – PDA(LR)):

Se parte desde la cadena de entrada y se identifican las reducciones a realizar para construir una derivación por la derecha de atrás hacia adelante. Genera así el árbol desde las hojas hacia la raíz.

Dada una gramática $GIC = (\Sigma_T, \Sigma_N, S, P)$ se debe construir un autómata con pila $AP = (\Sigma_T, \Sigma_T \cup \Sigma_N \cup \{\#\}, \{p, q, r\}, p, \#, \{r\}, f)$ de la siguiente forma:



PREANÁLISIS: Técnica de programación para hacer que el APND funcione de forma determinista. Consiste en “espiar uno o más símbolos de la entrada” sin leerlos (pre analizar), para decidir cuál de las producciones conviene utilizar frente a alternativas no deterministas que se presenten. Con esta técnica pueden desarrollarse algoritmos de análisis sintáctico que pueden programarse en forma eficiente:

- LL(k): Left to right read and leftmost derivation (ASD).
- LR(k): Left to right read and rightmost derivation (ASA).

Donde k es la cantidad de símbolos de entrada a pre analizar.

PROBLEMA DEL RETROCESO: Al igual que en los analizadores descendentes, la formulación básica de los ascendentes es esencialmente no determinista. Esto se origina en

la búsqueda del camino hacia la aceptación de la cadena y es conocido como el problema del retroceso: al existir varios caminos a seguir, se debe probar primero uno de ellos y si no se consigue el objetivo (aceptación) deberá volverse e intentarlo por otro.

Unidad N° 6: Autómatas Linealmente Acotados, Máquina de Turing y Complejidad

Autómata linealmente acotado y máquina de Turing

Definición

Definiciones El autómata linealmente acotado (ALA) queda definido como:

$$ALA = (\Sigma E, \Gamma, Q, q_0, A, f)$$

Donde:

Γ : Alfabeto de cinta $\Gamma = \Sigma E \cup \{ \vdash, \vdash \} \cup \Omega$

f : Función de transición, $f: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, N, D, P\}$

Funcionamiento

Siempre iniciando su operación desde el estado inicial q_0 con su cabezal sobre el símbolo de inicio de cinta el símbolo de más a la izquierda), el autómata:

1. Lee un símbolo de entrada e .
2. Graba un símbolo de salida s .
3. Transita a un estado q
4. Mueve el cabezal a Izquierda, Derecha, No lo mueve o Para.
5. Repite 1, 2, 3 y 4 hasta ejecutar una instrucción de Parada.
6. Al detenerse, decide si acepta o no la cadena de entrada.

- Durante su funcionamiento, el ALA no puede mover su cabezal a la izquierda si está leyendo BOT, y no puede mover su cabezal a la derecha, si está leyendo EOT.
- Podría quedar atrapado en un ciclo infinito y no terminar.
- Siempre para aceptar la cadena de entrada, se pedirá que al menos la haya leído una vez. Para eso pediremos que se detenga en un estado de aceptación con su cabezal en EOT.

Aceptación o reconocimiento de cadenas

Un ALA acepta o reconoce una cadena de símbolos de entrada $a \in \Sigma^*$, si puede detenerse (Parar) en un estado de aceptación $q_f \in A$ con su cabezal de lecto-escritura ubicado sobre el símbolo de fin de cadena EOT ($\vdash$).

$$(q_0, \vdash \alpha \dashv, 0) \vdash^* (q_f, \vdash \beta \dashv, |\beta| + 1), \text{ con } q_f \in A$$

Un ALA acepta o reconoce una cadena de símbolos de entrada $a \in \Sigma^*$, si puede moverse desde una configuración inicial a una configuración final de aceptación y detenerse (Parar). Si se tiene en cuenta la cadena que quedó en la cinta luego del procesamiento del ALA, puede verse a esta máquina como Ejecutora de Procedimientos: ALA: $\Sigma^* \rightarrow \Gamma^*$.

LENGUAJE RECONOCIDO $L(ALA)$: conjunto de cadenas de símbolos de entrada que son reconocidas por el ALA. Si se tiene en cuenta la cadena que quedó en la cinta luego del procesamiento del ALA, puede verse a esta máquina como Ejecutora de Procedimientos: ALA: $\Sigma^* \rightarrow \Gamma^*$.

Configuración o descripción instantánea

$$Kt = (qt, \vdash at \dashv, n)$$

$qt \in Q$: estado actual en el instante t

$at \in \Gamma^*$: contenido de la cinta en el instante t

$n \in N$: posición del cabezal sobre la cinta, $0 \leq n \leq |at| + 1$

Configuración Inicial: $K_0 = (q_0, \vdash \alpha_0 \dashv, 0)$.

Configuración Final: $K_n = (q_n, \vdash \alpha_n \dashv, m)$ con ALA detenido.

Si $q_n \in A$ y $m = |\alpha_n| + 1$, ésta ES una configuración de aceptación.

MOVIMIENTO: cambio de una configuración a otra:

$$(qt, \vdash at \dashv, nt) \vdash (qt+1, \vdash at+1 \dashv, nt+1)$$

Si es posible pasar de una a otra en una transición de f .

MOVIMIENTO GENERALIZADO: entre configuraciones:

$$(qt, \vdash at \dashv, nt) \vdash^* (qt+k, \vdash at+k \dashv, nt+k)$$

Si es posible pasar de una a otra en k transiciones de f .

Máquina de Turing

$$(\Sigma, \Gamma, Q, q_0, A, f, b)$$

Σ : Alfabeto de símbolos de entrada.

Γ : Alfabeto de símbolos de cinta. $\Gamma = \Sigma \cup \{b\} \cup \Omega$ donde b = blanco.

Q : Conjunto finito y no vacío de estados posibles.

q_0 : Estado inicial $q_0 \in Q$.

A : Conjunto de estados de aceptación $A \subseteq Q$.

f : Función de transición $f: Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, D, N, P\}$.

FUNCIONAMIENTO

Siempre iniciando su operación desde el estado inicial q_0 con su cabezal sobre el primer símbolo de la cadena de entrada, la máquina:

1. Lee un símbolo de entrada e .
 2. Graba un símbolo de salida s .
 3. Transita a un estado q .
 4. Mueve el cabezal a Izquierda, Derecha, No lo mueve o Para.
 5. Repite 1, 2, 3 y 4 hasta ejecutar una instrucción de Parada.
 6. Al detenerse, decide si acepta o no la cadena de entrada.
- Durante su funcionamiento, la MT puede modificar nuevas celdas de la cinta infinita llena con espacios en blanco si le es necesario para su tarea $\rightarrow$ memoria lineal infinita.
 - Podría quedar atrapada en un ciclo infinito y no terminar.
 - Puede aceptar una cadena sin tener que leerla completamente con sólo detenerse en un estado de aceptación.

ACEPTACIÓN O RECONOCIMIENTO DE CADENAS

Una MT acepta o reconoce una cadena de símbolos de entrada $a \in \Sigma^*$, si puede detenerse (Parar) en un estado de aceptación $q_f \in A$. Una MT acepta o reconoce una cadena de símbolos de entrada $a \in \Sigma^*$, si puede moverse desde una configuración inicial a una configuración final de aceptación y detenerse (Parar).

$$(q_0, b \alpha b, 0) \vdash^* (q_f, b \beta b, m), \text{ con } q_f \in A$$

Si se tiene en cuenta la cadena que quedó en la cinta luego del procesamiento de la MT, puede verse a esta máquina como Ejecutora de Procedimientos: MT: $\Sigma^* \rightarrow \Gamma^*$.

LENGUAJE RECONOCIDO $L(MT)$: conjunto de cadenas de símbolos de entrada que son reconocidas por la MT.

CONFIGURACIÓN O DESCRIPCIÓN INSTANTÁNEA

$$Kt = (qt, b \alpha b, n)$$

$qt \in Q$: estado actual en el instante t

$\alpha t \in \Gamma^*$: contenido de la cinta en el instante t
 $n \in \mathbb{Z}$: posición del cabezal sobre la cinta, $-\infty < n < +\infty$

Configuración Inicial: $K0 = (q0, b, \alpha 0 b, 1)$

Configuración Final: $Kn = (qn, b, \alpha n b, m)$ con MT detenida.

Si $qn \in A$, esta ES una configuración de aceptación.

MOVIMIENTO: cambio de una configuración a otra:

$$(qt, b, \alpha t b, nt) \vdash (qt+1, b, \alpha t+1 b, nt+1)$$

Si es posible pasar de una a otra en una transición de f .

MOVIMIENTO GENERALIZADO: entre configuraciones:

$$(qt, b, \alpha t b, nt) \vdash^* (qt+k, b, \alpha t+k b, nt+k)$$

Si es posible pasar de una a otra en k transiciones de f .

Variantes

Modularizada

La idea es crear pequeñas máquinas de Turing que realicen funciones sencillas, para luego combinarlas unas con otras, para “programar” problemas más complejos, de manera más sencilla.

Universal

Máquina de Turing que puede leer otra MT convenientemente codificada en su cinta y la entrada de ésta, para emular su funcionamiento.

Generalizada

Máquinas con más hardware: más de una cinta, más cabezales, cintas multidimensionales, pilas, colas, contadores, etc.

No determinista

MT con función $f: Q \times \Gamma \rightarrow P(Q \times \Gamma \times \{I, D, N, P\})$. Con esta función podría en cada paso hacer varias cosas distintas en forma simultánea.

En todos los casos siempre se ha encontrado una MT determinista tradicional de una sola cinta EQUIVALENTE.

MÁQUINA DE TURING CON ORÁCULOS: En su tesis doctoral, dirigida por Alonzo Church, Alan Turing presenta una máquina que al no poder decidir qué hacer, llama a una función (el

oráculo) que devuelve una predicción cero o uno, y basada en esa información continúa. Esta máquina resuelve el problema de la detención.

Complejidad Computacional

Concepto

Se asocia el concepto de complejidad con los recursos de cómputo requeridos para resolver un problema.

El foco de la teoría de la computabilidad (TC) está en la existencia o no de procedimientos que permitan resolver cierto problema.

La teoría de la complejidad computacional (TCC) es la que se ocupa del esfuerzo asociado para completar ese cómputo.

El concepto de complejidad no parece ser cuantificable o medible, sino más bien subjetivo.

Complejidad de los problemas y de las soluciones

Clases de problemas

Clase P: Incluye los problemas que pueden ser resueltos en tiempo polinómico por una MTD. Los problemas de complejidad temporal polinómica son denominados tratables.

Clase NP: Incluye los problemas que pueden ser resueltos por una MTND en tiempo polinómico, lo que en la práctica conduce en general a una complejidad exponencial. Estos problemas son denominados intratables.

NP-completos: un subconjunto problemas difíciles de NP, a los que todos los otros problemas NP pueden reducirse en tiempo polinómico.

Métricas: complejidad estructural, espacial y temporal de la máquina de Turing

Una de las primeras medidas de complejidad para máquinas de Turing fue la propuesta por Shannon en 1956, que apuntaba a la complejidad estructural de la máquina y proponía como parámetro al producto del número de estados por el número de símbolos de cinta: $|Q| \cdot |\Sigma|$. Este enfoque, despertó gran interés en el ámbito de la simulación, donde se destacan los siguientes teoremas:

- **Teorema 1 de Shannon:** Cualquier máquina de Turing con símbolos y n estados puede ser simulada por otra máquina de Turing con exactamente dos estados y $4 \cdot m \cdot n + m$ símbolos. En particular, existe una máquina universal de Turing de solo dos estados. Además, toda máquina universal de Turing necesita al menos dos estados.

- **Teorema 2 de Shannon:** Cualquier máquina de Turing con símbolos y n estados puede ser simulada por otra con exactamente dos símbolos y menos de $8 \cdot m \cdot n$ estados.

La **complejidad estructural** propuesta por Shannon no conduce a la determinación de la complejidad operativa de la máquina, por lo que esta propuesta es poco utilizada. Como alternativa, se propuso determinar indirectamente la complejidad de los problemas a partir de la medición de los recursos necesarios para resolverlos. Esta nueva propuesta se apoyó en la obvia convicción que resolver un problema de elevada complejidad demandará más recursos que resolver un problema sencillo. Es así que a partir de entonces se adoptaron el tiempo y el espacio como los indicadores más representativos de la complejidad de los problemas.

El primer indicador se refiere a la cantidad de intervalos o unidades elementales de tiempo que demanda completar una computación, es decir, la ejecución de un proceso. Este indicador es definido como **Complejidad Temporal $T(n)$** del problema y es expresado en función del tamaño n del problema o de sus datos.

El segundo indicador es la cantidad de **espacio de almacenamiento** requerido para resolverlo y se apoya en la idea de que al aumentar la complejidad de un proceso aumenta también el espacio que su solución demanda. Este parámetro es reconocido como la **Complejidad Espacial, $E(n)$** del problema y también es expresado en función de n .

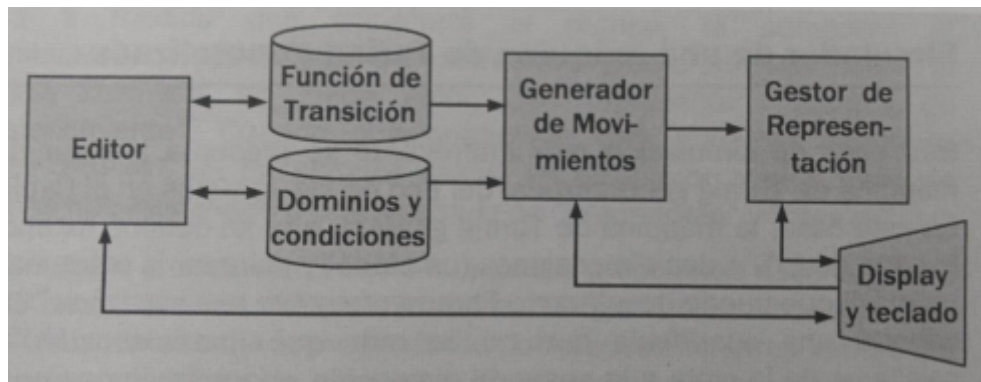
Unidad Nº 7: Simuladores de Máquinas Abstractas

Simulación y otros conceptos relacionados

En las ciencias, la simulación es el artificio contextual que hace referencia a la investigación de una hipótesis o un conjunto de hipótesis de trabajo utilizando modelos un método perfecto para la enseñanza y aprendizaje.

simulador hace referencia a una herramienta o producto final específicamente diseñado para recrear un sistema, un fenómeno o una máquina abstracta, usualmente con el fin de analizar sus características y estudiar su comportamiento

Arquitectura de los simuladores



Editor

Está destinado a la carga y actualización de dominios, función de transición y condiciones iniciales de operación (requerimientos funcionales a, b, c, y d; requerimientos no funcionales a, b, c, y d; finalmente requerimientos de interfaz gráfica a).

Generador de movimientos

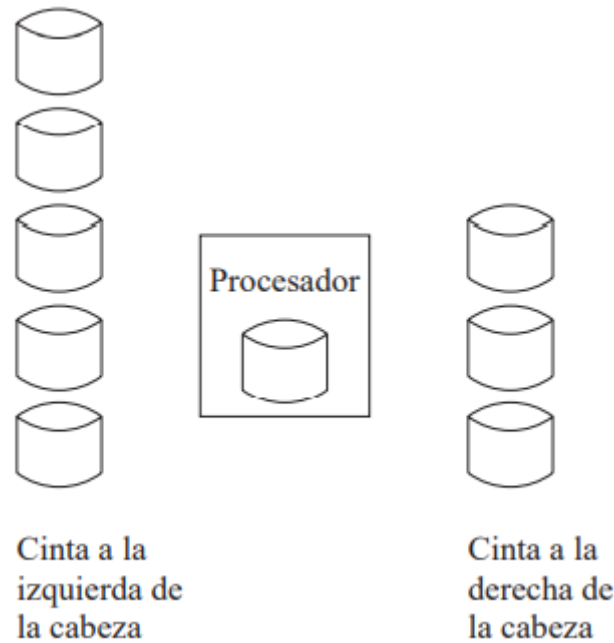
Representa el núcleo operativo del sistema, su misión es ejecutar los movimientos de las máquinas de estados y hacer las actualizaciones en la configuración según corresponda a cada tipo de autómeta (requerimientos funcionales e, f, y g; requerimientos no funcionales a, c, y d).

Gestor de representación

Tiene a cargo la interfaz gráfica con el usuario durante la operación de la máquina abstracta, mostrar su evolución y poner a su disposición los comandos que puedan corresponder en cada caso (requerimientos no funcionales a y b; requerimientos de interfaz gráfica b, c, d, e, f y g).

Simulación de Máquinas Abstractas

Examinemos en primer lugar cómo una computadora puede simular una máquina de Turing. Dada una MT concreta M , debemos escribir un programa que actúe como M . Un aspecto de M es su unidad de control. Dado que sólo existen un número finito de estados y un número finito de reglas de transición, nuestro programa puede codificar los estados como cadenas de caracteres y utilizar una tabla de transiciones, que consultará para determinar cada movimiento. Del mismo modo, los símbolos de cinta pueden codificarse como cadenas de caracteres de longitud fija, ya que el número de estos símbolos es finito. Al estudiar cómo simulará el programa la cinta de la máquina de Turing surge una cuestión importante. Esta cinta puede crecer en longitud infinitamente, pero la memoria de una computadora (memoria principal, discos y otros dispositivos de almacenamiento) es finita. ¿Podemos simular una cinta infinita con una cantidad fija de memoria? Si no existe la posibilidad de reemplazar los dispositivos de almacenamiento, entonces realmente no podremos; la computadora entonces se comportaría como un autómata finito y los únicos lenguajes que aceptaría serían los regulares. Sin embargo, las computadoras comunes disponen de dispositivos de almacenamiento intercambiables, por ejemplo, discos “Zip”. De hecho, el típico disco duro es un dispositivo extraíble y puede reemplazarse por otro disco idéntico pero vacío. Dado que no existe un límite evidente en lo que se refiere a cuántos discos podríamos utilizar, suponemos que podemos disponer de tantos discos como necesite la computadora. Podemos por tanto disponer que los discos se coloquen en dos pilas, como se muestra en la Figura 8.21. Una pila almacena los datos de las casillas de la cinta de la máquina de Turing que están situadas a la izquierda de la cabeza de la cinta y la otra pila almacena los datos almacenados a la derecha de la cabeza de la cinta. Cuanto más profundo se encuentre un símbolo en una pila, más alejado de la cabeza de la cinta se encontrará el dato. Si la cabeza de la cinta de la MT se alejara hacia la izquierda lo suficiente como para llegar a casillas que no están representadas en el disco actualmente montado en la computadora, entonces se imprimirá el mensaje “cambiar izquierda”. En este caso, un operador cambiaría el disco que está montado por otro y lo colocaría encima de la pila de la derecha. El disco situado en la parte superior de la pila de la izquierda se monta en la computadora y se reanuda el proceso. De manera similar, si la cabeza de la cinta de la MT alcanza casillas suficientemente alejadas por la derecha que no están representadas por el disco montado actualmente, entonces se imprime el mensaje “cambiar derecha”. El operador retirará entonces el disco actualmente montado en la parte superior de la pila izquierda y montará el disco en la parte superior de la pila derecha de la computadora. Si cualquiera de las pilas está vacía cuando la computadora solicita que se monte un disco de dicha pila, quiere decir que la MT ha entrado en una zona de todo blancos de la cinta. En este caso, el operador deberá acudir al almacén y adquirir un disco vacío para instalarlo.



Selección y evaluación de simuladores

La selección y evaluación de simuladores la haremos a través de los siguientes requerimientos:

Requerimientos funcionales de un SMA

- 1) Definir los dominios de operación, representados por el alfabeto de entrada, alfabeto de cinta, alfabeto de pila y alfabeto de estados según sea el tipo de máquina.
- 2) Editar, almacenar y recuperar la función de transición, que será definida como una tabla.
- 3) Definir las condiciones iniciales de operación, que incluyen la cadena de entrada, el estado inicial, los estados finales y las posiciones de cabezales de lectura/escritura en caso de que correspondan.
- 4) Verificar la consistencia entre los dominios de operación, la función de transición y las condiciones iniciales de operación.
- 5) Ejecutar, a partir de la configuración inicial, los movimientos necesarios hasta alcanzar una configuración final, que podrá ser de aceptación o rechazo.
- 6) Admitir dos modos de operación: 1. Completa: que cumplirá todos los movimientos necesarios desde la configuración inicial a la final; 2. Paso a paso: donde cada paso implica la ejecución de un solo movimiento y es activado mediante un comando del sistema.
- 7) Determinar métricas o indicadores de operación, complejidad y/o eficiencia con el fin de facilitar la evaluación del desempeño de las máquinas y las comparaciones entre ellas.

Requerimientos no funcionales de un SMA

- 1) **Portabilidad:** El programa debe ser cross-plataform.
- 2) **Facilidad de uso:** La operación del SMA debe ser simple, amigable y natural. En lo posible con una ventana de ayuda.

3)**Robustez:** El SMA debe estar preparado para reconocer el ingreso de datos incorrectos, lo cual dará lugar a advertencias del caso y la no autorización del inicio del proceso de simulación. Se debe asegurar que estos errores no serán nunca causa de fallo. También se debe detectar la eventualidad de la repetición de una misma configuración, que sería un indicador de haber entrado en un ciclo infinito.

4)**Eficiencia:** Se trata de un requerimiento esencial de los procesos de simulación que involucran cálculo numérico intensivo, lo cual no es el caso de los SMA, sin embargo, la eficiencia debe ser considerada en el caso de la simulación de autómatas no deterministas con elevado factor de ramificación en sus árboles de descripciones instantáneas. La eficiencia también abarca a las actividades auxiliares como son la definición de los dominios y condiciones de operación.

Interfaz gráfica (GUI)

- 1)Facilidad para la edición de dominios, función de transición y condiciones iniciales de operación.
- 2)Representación de los componentes formales del autómata a lo largo de todo el proceso de simulación.
- 3)Representación de la configuración del autómata en cada intervalo de tiempo.
- 4)Opción de Debugging que permita alterar el contenido de la cinta de entrada o entrada/salida según el caso durante la operación del autómata.
- 5)Representación de un histórico de los sucesivos movimientos desde la configuración inicial a la final.
- 6)Representación gráfica del árbol de configuraciones o descripciones instantáneas.
- 7)Opción para la visualización de los indicadores de operación .

Unidad N° 8: Introducción a la semántica de lenguajes

Semántica de Lenguajes

Concepto

Conjunto de reglas que especifican el significado de toda sentencia sintácticamente correcta, escrita en el mismo. En el caso de los lenguajes de programación, existen dos formas básicas que permiten describir sus contenidos semánticos: la especificación natural y la especificación formal. La semántica de un lenguaje de programación estará representada por el conjunto de reglas que especifican el significado de toda sentencia sintácticamente correcta del lenguaje. El componente de software que debe realizar estos análisis sobre un programa fuente escrito en un lenguaje de programación es el COMPILADOR.

Semántica natural

Uso del lenguaje natural para explicar el significado de los símbolos, expresiones y sentencias del lenguaje.

Especificación formal

Definir el significado de las sentencias de un programa y de los programas completos mediante una formulación al estilo matemático. Se necesita especificación formal, entre otras cosas para:

- a) **Diseño de nuevos lenguajes de programación**, registrando decisiones sobre construcciones particulares y facilitando la identificación de errores u omisiones.
- b) Necesidad de **estandarización de los lenguajes** mediante la publicación de su semántica de un modo universalmente reconocible.
- c) **Diseño e implementación de los procesadores** de los nuevos lenguajes (compiladores o intérpretes) y de los generadores de compiladores.
- d) Necesidad de **identificar posibles ambigüedades** en las implementaciones de procesadores de lenguajes de programación o en sus documentos descriptivos.
- e) **Comprensión de los lenguajes** por parte de los programadores.
- f) **Verificación de propiedades de programas** a través de pruebas de corrección (verificación) o de la información relacionada con su ejecución.

Operacional

Este es el enfoque más antiguo e intuitivo. Consiste en describir el comportamiento de las sentencias del programa, indicando qué secuencia de operaciones o cálculos deben ejecutarse para implementar lo que se quiere que la sentencia haga (el cómo). Esto puede

hacerse asociando a cada a cada regla sintáctica, código de máquina (real o abstracta) o un conjunto de reglas de inferencia lógica que permita ir haciendo una deducción.

Denotacional y axiomática

La idea es describir el significado de una sentencia por una función, actuando sobre algún conjunto (con elementos como variables, memoria, tipos, etc.) produzca como salida el resultado que debería dar la sentencia (el qué). Se hace hincapié en el resultado de la ejecución y no en cómo obtenerlo.

METALENGUAJES QUE DEFINEN PROPIEDADES:

AXIOMÁTICA: Se pretende describir el lenguaje de programación como un Sistema Axiomático (términos primitivos, axiomas, definiciones) que permita obtener el resultado de la ejecución del programa como una deducción desde los datos de entrada (un teorema). También esto permitiría inferir y/o verificar ciertas propiedades de los programas que se crean necesarias (terminación, corrección , equivalencias, etc.).

ALGEBRAICA: El lenguaje de programación se presenta como una estructura algebraica (elementos, conjuntos y funciones), orientada a definir los tipos y estructuras de datos como conjuntos de valores, y las operaciones que son posibles entre ellos. Es equivalente al enfoque axiomático pero basado en álgebra; se acerca al enfoque Denotacional , ya que intenta describir el significado de las sentencias como el resultado de aplicar esas operaciones a las estructuras de datos.

DE ACCIONES: La definición de la semántica del lenguaje está basada en un conjunto de acciones, que son las operaciones habitualmente previstas en los lenguajes de programación. Con este fin, se definen primitivas para la asignación y declaración de variables y la combinación de instrucciones mediante control de flujo secuencial, condicional e iterativo donde se establecen las propiedades que su aplicación verifica sobre las variables. Se acerca al enfoque operacional.

Gramáticas con atributos

Las gramáticas atribuidas son la forma habitual de especificar la semántica de los lenguajes de programación comerciales. Agrega a la gramática independiente del contexto del lenguaje un conjunto de propiedades de los símbolos gramaticales (los atributos de esos símbolos de $\Sigma_T \cup \Sigma_N$), reglas para determinar los valores de esas propiedades y restricciones que se imponen a las propiedades y a su cálculo. Este enfoque permite hacer buena parte del análisis semántico guiado por la sintaxis que está definida en la gramática formal del lenguaje.

Las gramáticas con atributos (GA) o gramáticas atribuidas deben su nombre a que se apoyan en la asignación de atributos (propiedades) a las distintas construcciones sintácticas de un lenguaje. Las gramáticas son de tipo 2 (independientes de contexto) a las que se incorporan atributos a sus símbolos terminales y no terminales, las reglas para su evaluación y las condiciones que éstas deben cumplir.

Una gramática con atributos es una gramática de contexto libre a la que se le ha incorporado un conjunto de atributos (A), un conjunto de reglas semánticas (R) y un conjunto (C) de condiciones, donde los elementos de A están relacionados con los símbolos terminales y no terminales, mientras que los de R y C están asociados a cada una de las reglas de producción de P :

$$GA = (\Sigma T, \Sigma N, S, P, A, R, C)$$

- a. Los primeros cuatro componentes son los que corresponden a toda gramática formal. Las reglas de producción deben responder a las condiciones que son propias de una gramática independiente de contexto y pueden estar expresadas en formas normales, en las que las de Greinbach y de Chomsky son las más habituales.
- b. Denominados $\Sigma = \Sigma T \cup \Sigma N$, el quinto componente de la GA es identificado como $A(\Sigma)$, representando al conjunto de atributos asociados a los símbolos del alfabeto Σ . Cada atributo se denota con un nombre precedido por un punto y el nombre del símbolo al que está asociado.
- c. A cada regla de producción del conjunto P , se le asocia un número finito de acciones o reglas semánticas que especifican la forma en que se modifican los atributos con la aplicación de la regla sintáctica. Al conjunto de todas las reglas semánticas se lo define como $R(P)$, que es el sexto componente de la GA.
- d. El séptimo y último componente de la GA es conjunto $C(P)$ de condiciones asociado a cada una de las reglas de producción P . Estas condiciones, que deben ser cumplidas por lo valores de los atributos, describen un sublenguaje del lenguaje definido por la sintaxis de la gramática.

Atributos

Dada una **GIC** = (ΣT , ΣN , S , P) llamaremos atributo a una propiedad asignable a los símbolos gramaticales de la gramática ($\Sigma = \Sigma T \cup \Sigma N$) que tendrá un nombre y un valor de un conjunto de posibles valores bien definido.

Los atributos representan propiedades de los símbolos terminales y no terminales de una gramática y su naturaleza varía según la información que contienen y la fase de procesamiento en la que se encuentren.

Por ejemplo, el tipo de una variable, su valor, su dirección asignada en memoria, etc.

NOTACIÓN: si $a \in \Sigma$ y XYZ es un atributo de a , se denotará $a.XYZ$

EJEMPLOS: Supongamos que un identificador ID es un símbolo gramatical y que puede tener como atributos un nombre, una dirección de memoria, un valor contenido en ella y un tipo de dato que puede contener; entonces los podríamos denotar ID.nombre , ID.dirección , ID.value , ID.tipo.

Reglas semánticas

Dependiendo del tipo de instrucción, las acciones semánticas que están incluidas en el conjunto de reglas $R(P)$ tienen distintas finalidades y pueden agruparse según se presenta en la siguiente tabla:

TIPO DE INSTRUCCIÓN	ACCIONES SEMÁNTICAS
Declaración	Obtención de tipos de la tabla de símbolos
Ejecutables	Completar la definición de tipos de los símbolos
Funciones y procedimientos	Comprobación del número, orden y tipo de argumentos
Identificación de variables	Comprobación de identificación previo a su uso
Etiquetas	Comprobación de repetición y validación
Constantes	Comprobación de intentos de alteración de valores
Conversiones	Verificación de equivalencias
Sobrecarga de operadores	Identificación y resolución

Atributos heredados y sintetizados

Son aquellos que se clasifican según la **forma** de calcular el valor con el árbol de derivación:

Atributo sintetizado: utilizando el valor de los atributos de los nodos hijos a los del no terminal en un nodo $AS(\Sigma)$: ASA

Atributos heredados: utilizando el valor de los atributos del nodo padre y de los hermanos (aquellos a la izquierda en la producción al nodo en cuestión) $AH(\Sigma)$: ASD .

Atributos estáticos y dinámicos

Según el **MOMENTO** en el cual se determinan sus valores:

Atributo estático: en tiempo de compilación.

Atributo dinámico: en tiempo de ejecución.

Cuadro resumen máquinas abstractas

[Cuadro resumen](#)

Práctico

Gramáticas FNC y FNG

Minimización de autómatas

Anexo: Bibliografía y Links

Modalidad Académica

[Modalidad Académica 2022](#)

Bibliografía

Para todas las unidades, el material obligatorio es el mismo, estos son:

- Giró J. F., Vázquez J. C., Meloni B. E., Constable L. E. (2015). Lenguajes Formales y Teoría de Autómatas – Capítulo 8. Buenos Aires, Argentina. Alfaomega.